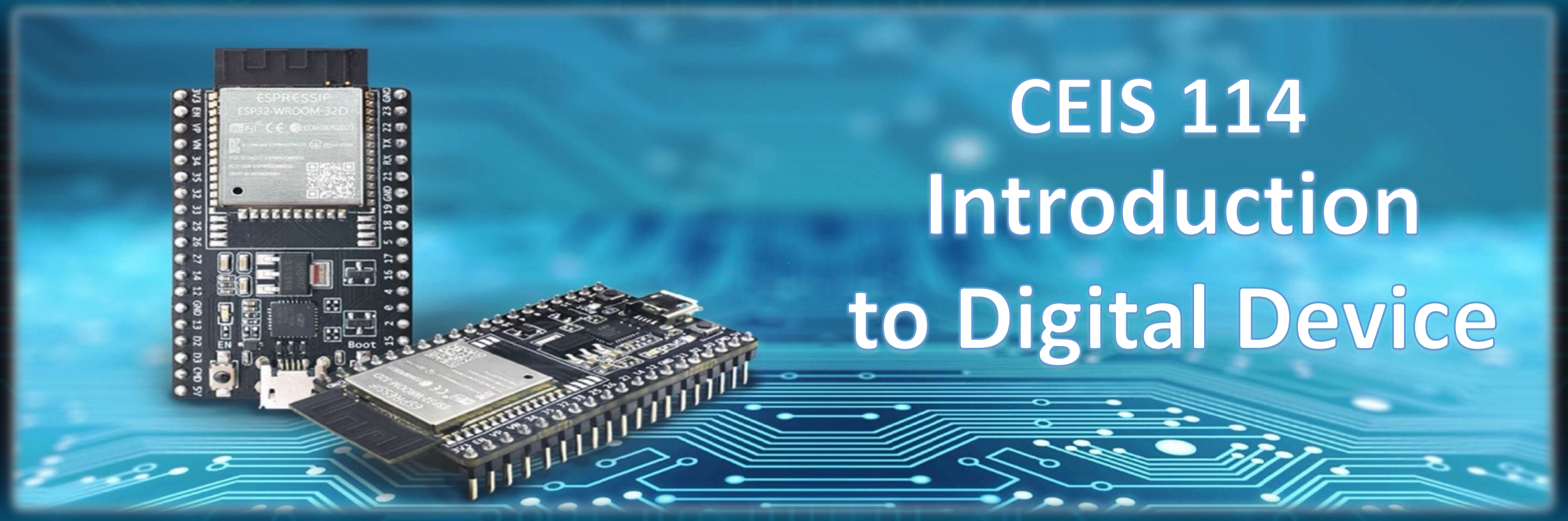
Creation of an IoT device that simulates a multiple traffic controller system by employing an ESP32 Microcontroller, a Web server, and a Smart Sensor.



CEIS 114
Introduction to Digital Device

Mildred Owens
Cybersecurity,
Devry University Online

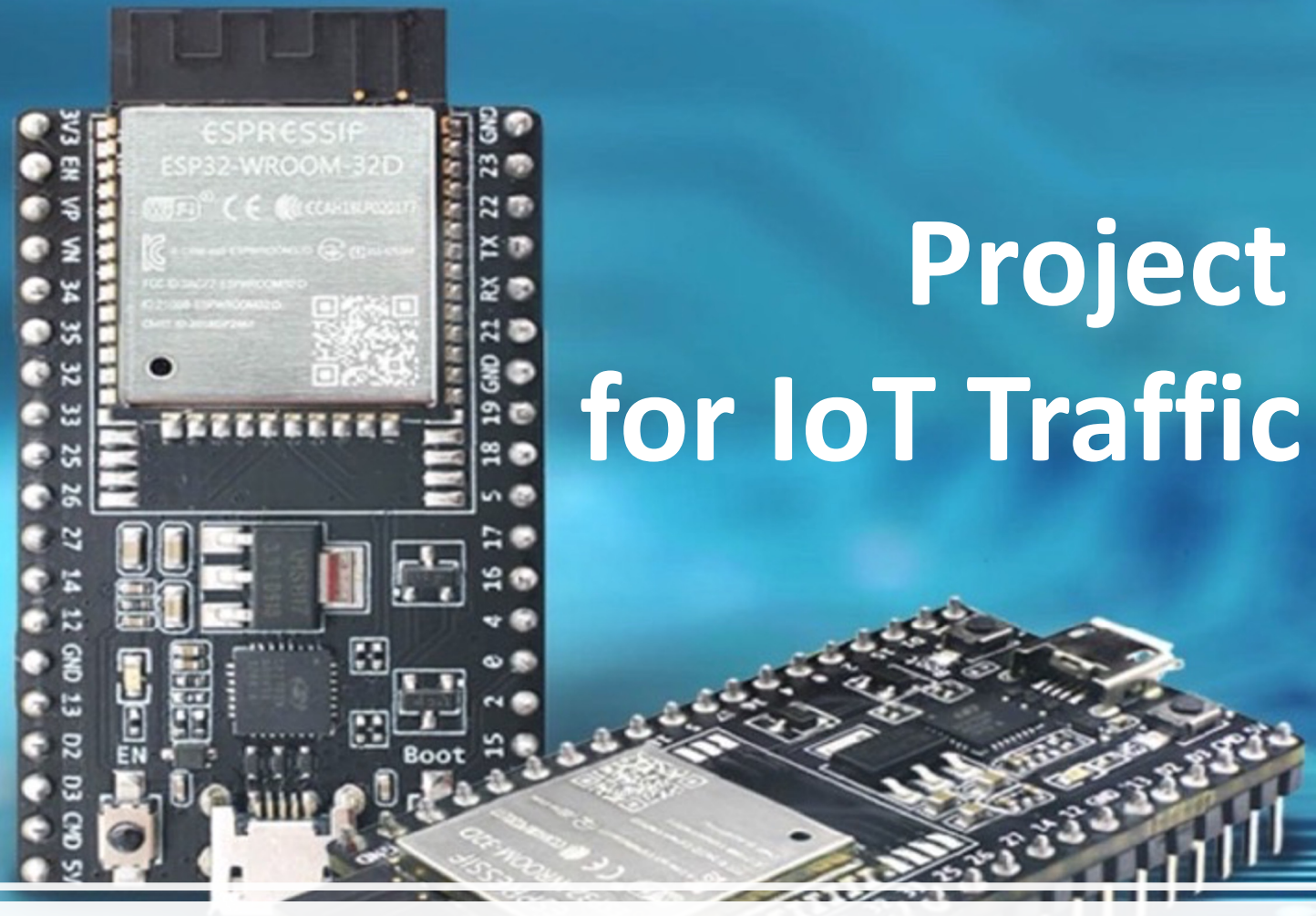
INTRODUCTION

The Internet of Things (IoT) is rapidly expanding. New technologies and applications are constantly being invented, resulting in a plethora of new work prospects, each with its own set of required skill sets.

It is critical to keep current with the skills necessary for an everchanging employment market and to take advantage of the numerous accessible learning opportunities.

- This course project integrates hardware, software, and networks into a complete system to cover the core notion of the Internet of Things.
- The project focuses on the creation of an IoT device that simulates a multiple traffic controller system by employing an ESP32 Microcontroller, a Web server, and a Smart Sensor.
- The device's design and development process comprises planning, hardware setup, programming, and networking. It covers several elements of IoT and will prepare you for a profession in technology in the future.

Project Plan for IoT Traffic Controller

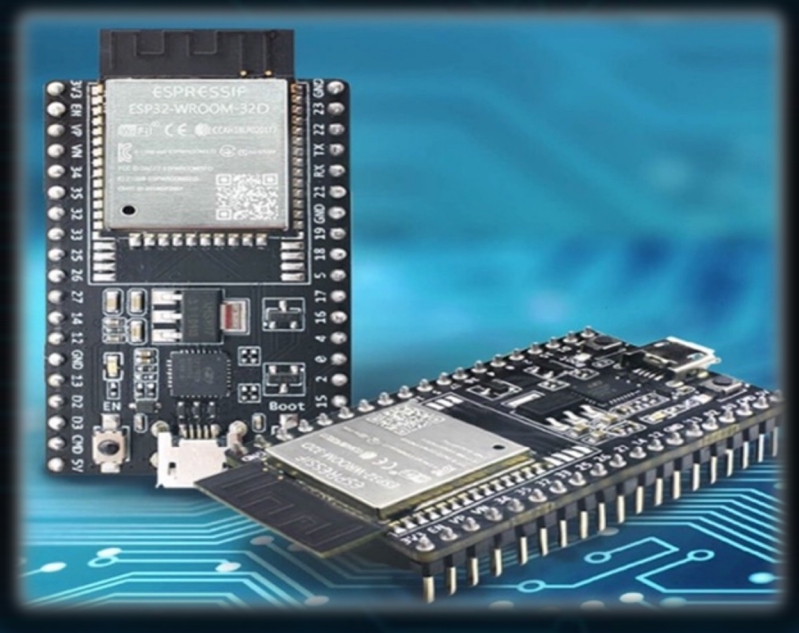


MODULE 2

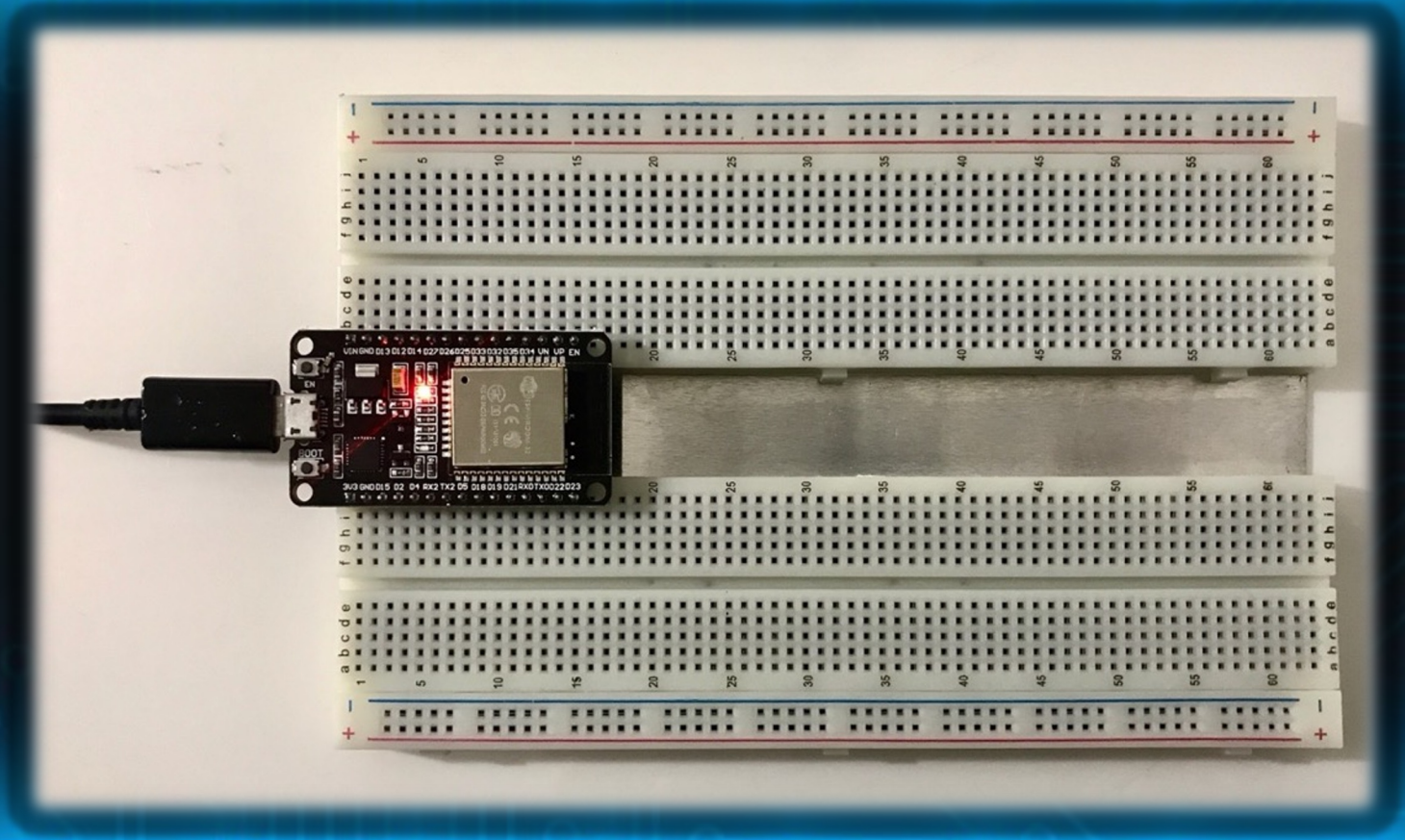
Inventory

- ESP 32 Board
- Colored LEDs: 2 sets
- Red, Yellow, Green, Blue
- 220 Ohm Resistors (optional)
- Wires
- Breadboard(s)
- LCD Unit with I2C Adapter
- Active Buzzer
- Mini Router
- Push Button(s)
- PIR Motion Sensor

Microcontroller mounted and powered ON

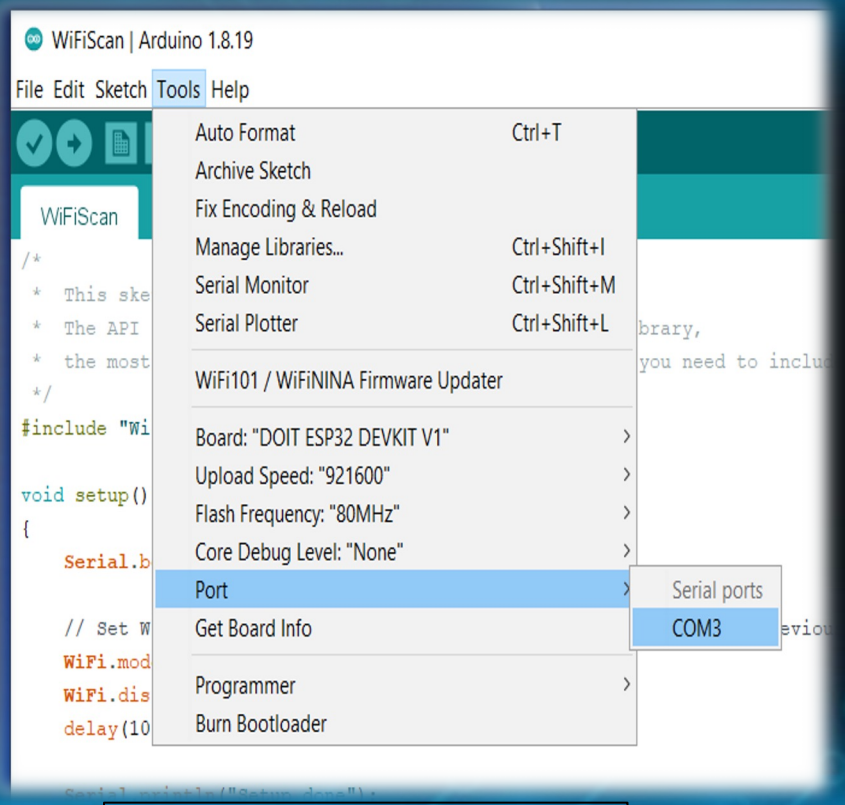


ESP32

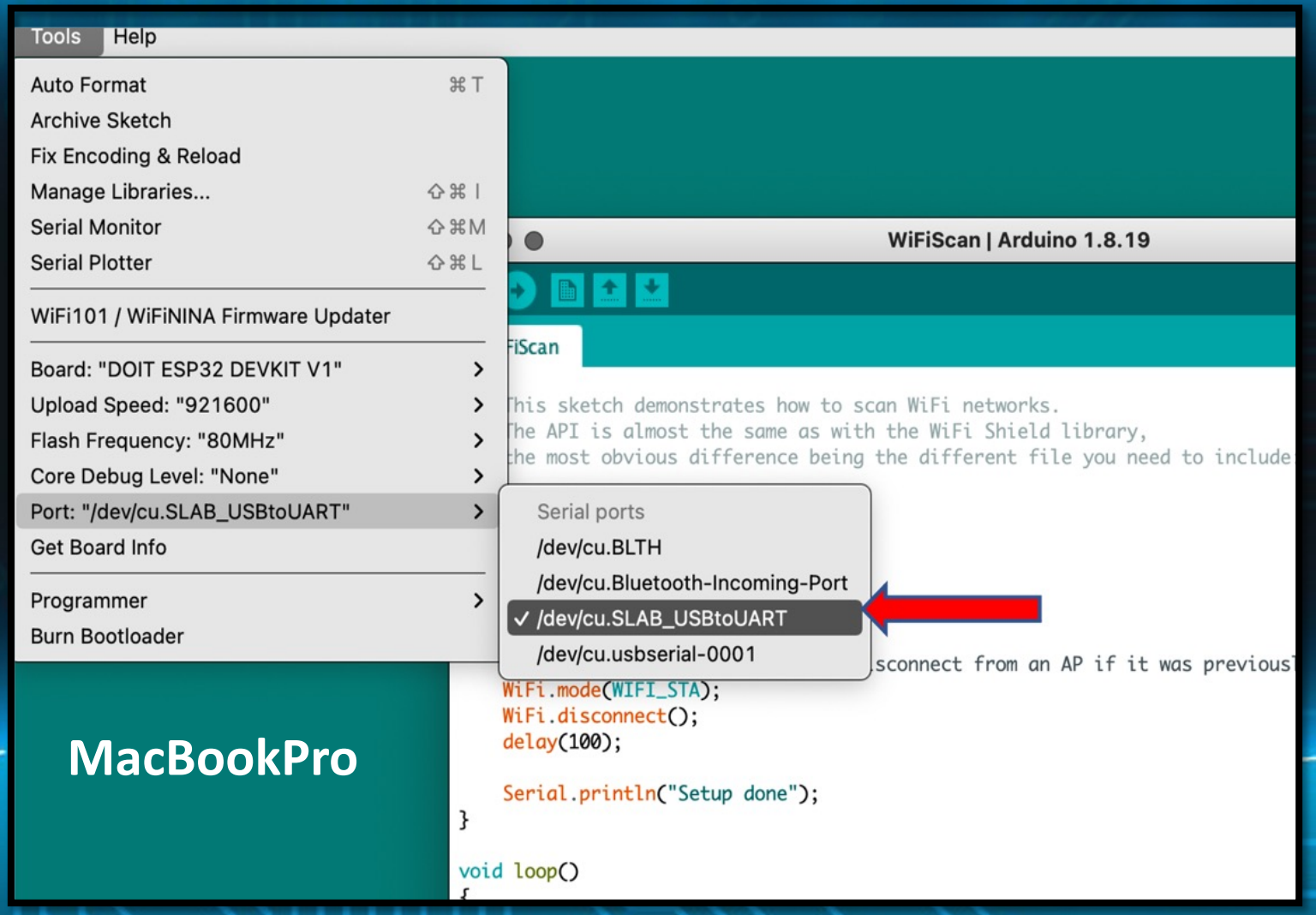


Installation On Arduino IDE

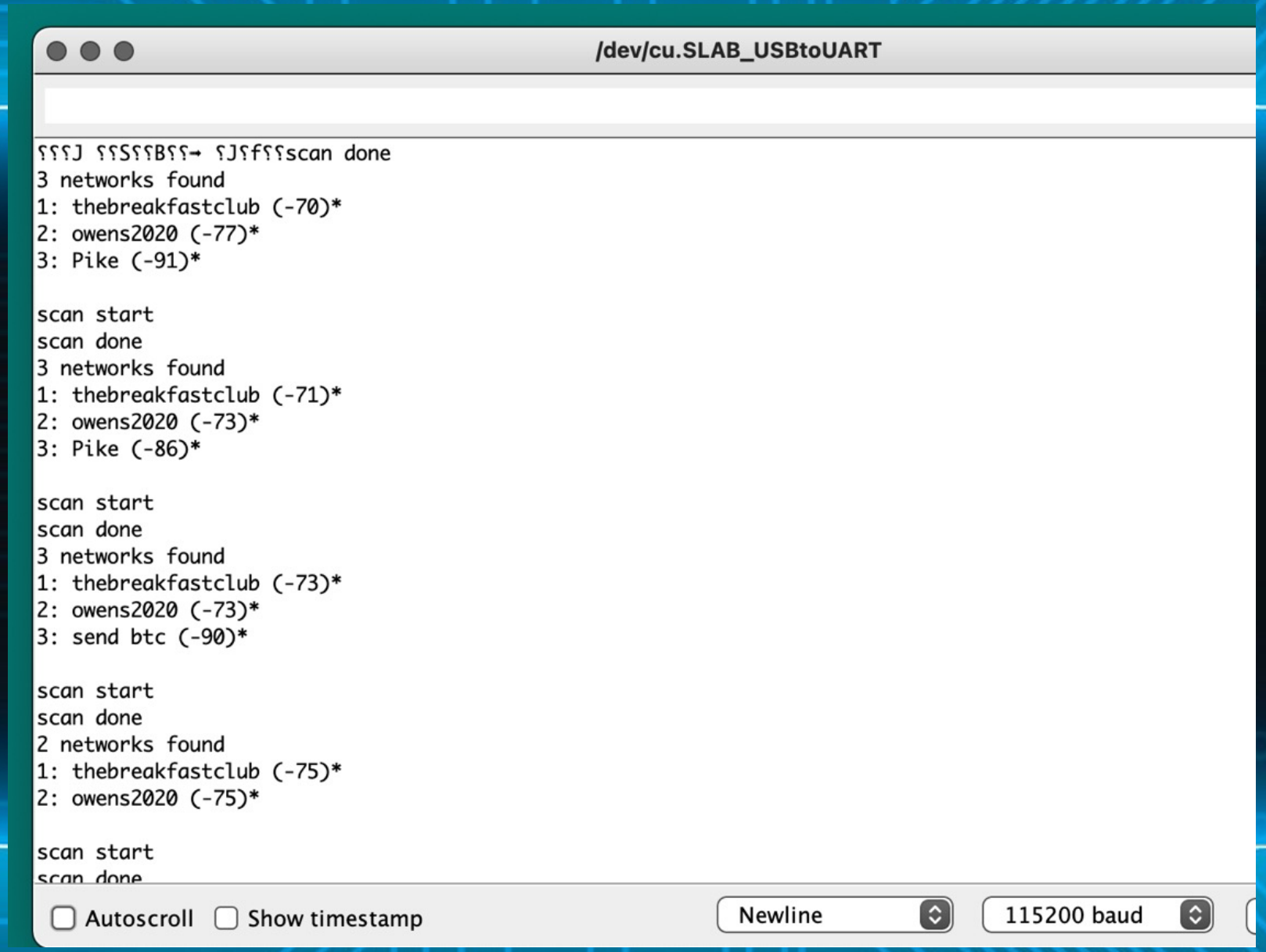
with Port selected from Tools menu On Arduino IDE



Using Windows 10

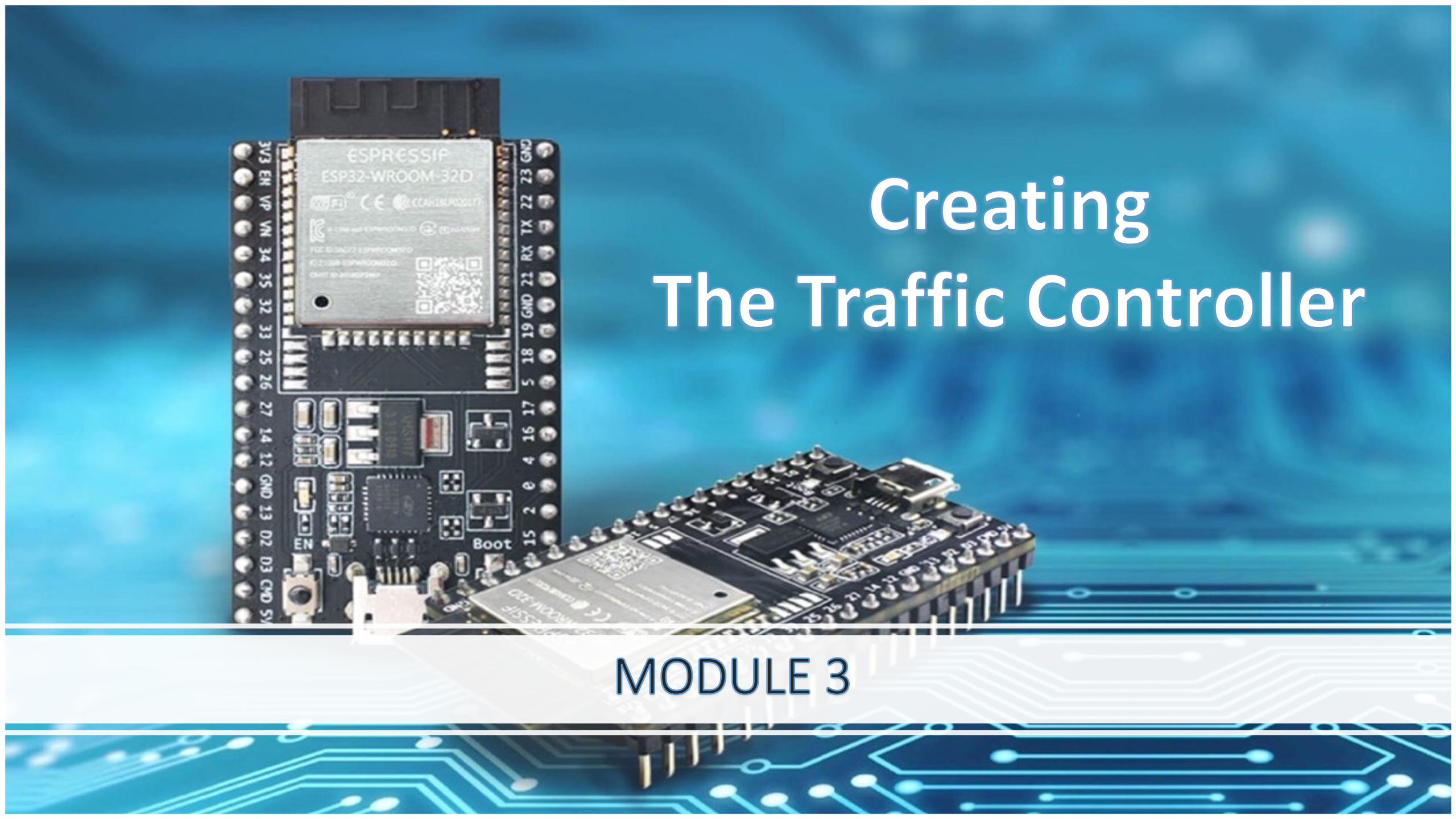


MacBookPro



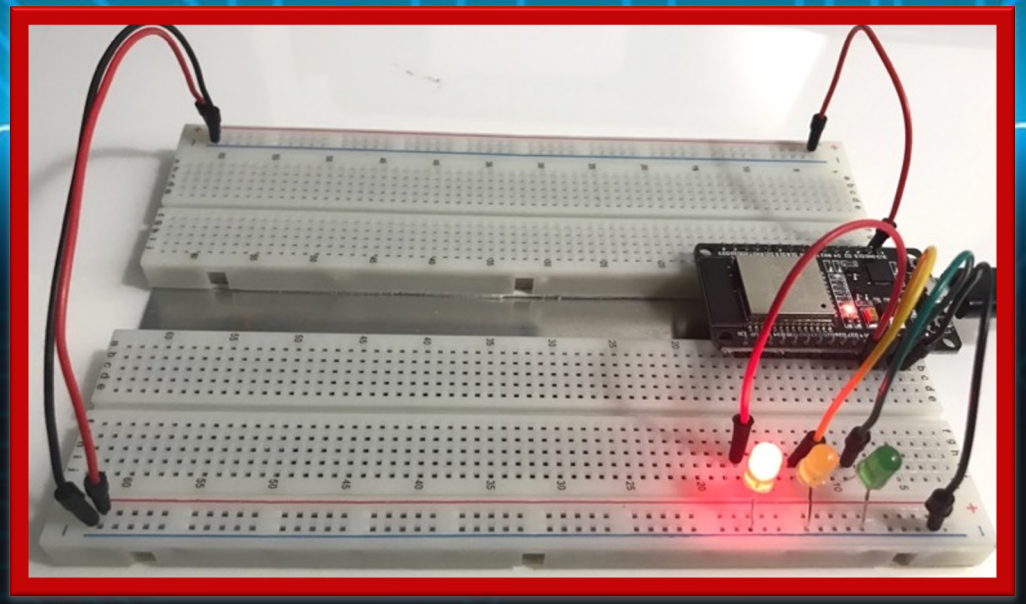
- **ESP32 WiFi Scan**
- **On Serial Monitor Arduino IDE**

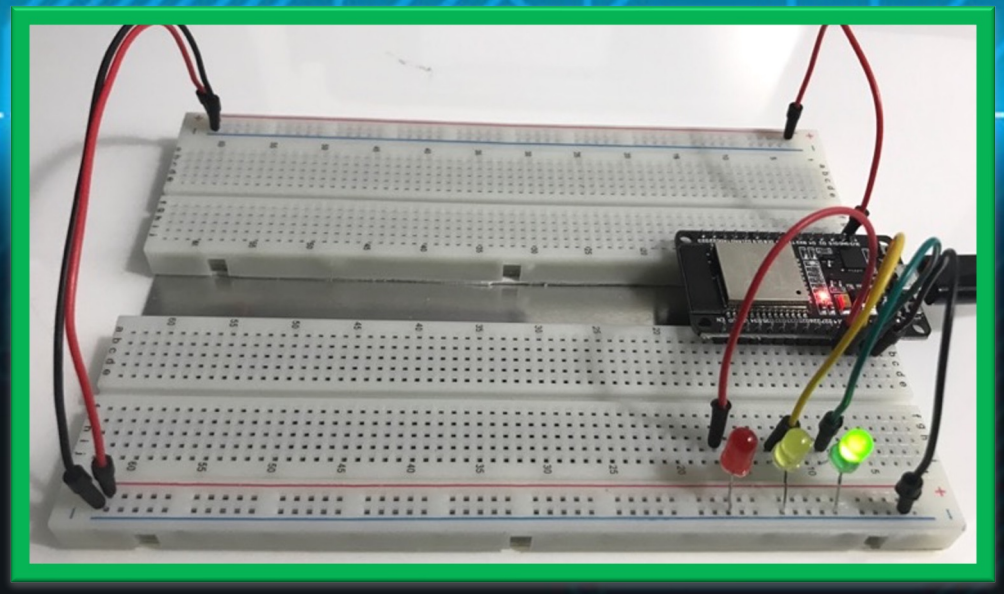
- ***showing the available networks***

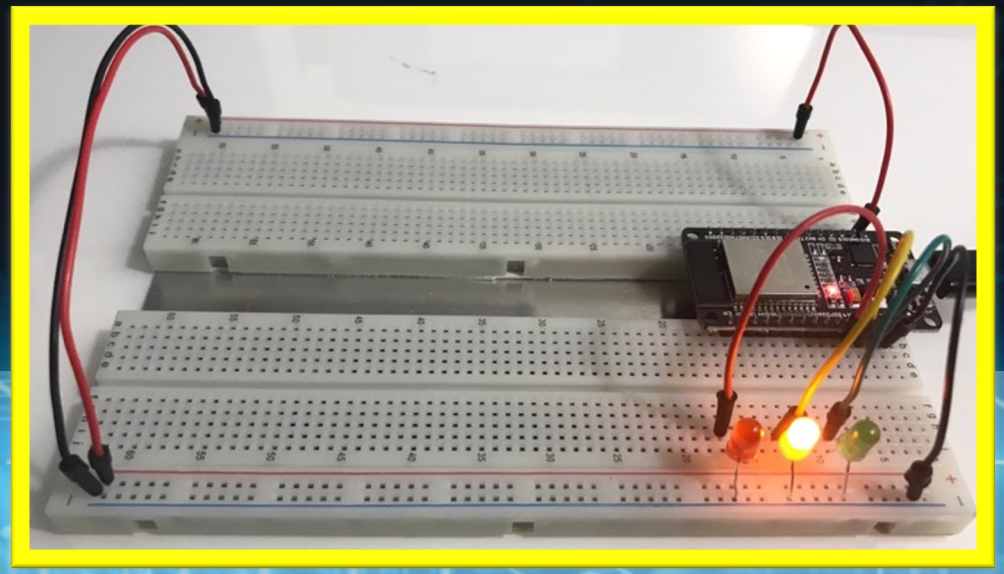


Creating The Traffic Controller

MODULE 3

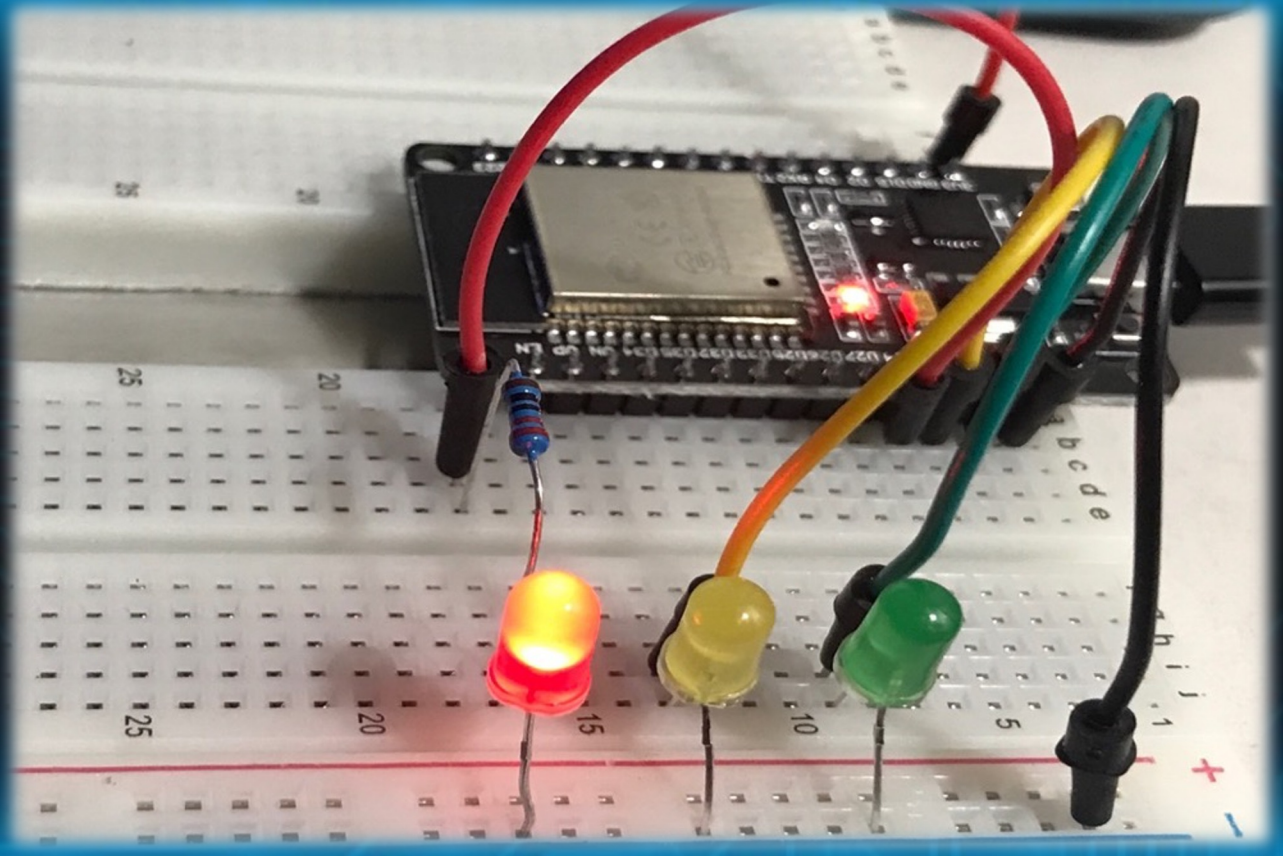






- ESP 32 Board
- Colored LEDs: Red, Yellow and Green
- 220 Ohm Resistors (optional)
- Wires
- Breadboard

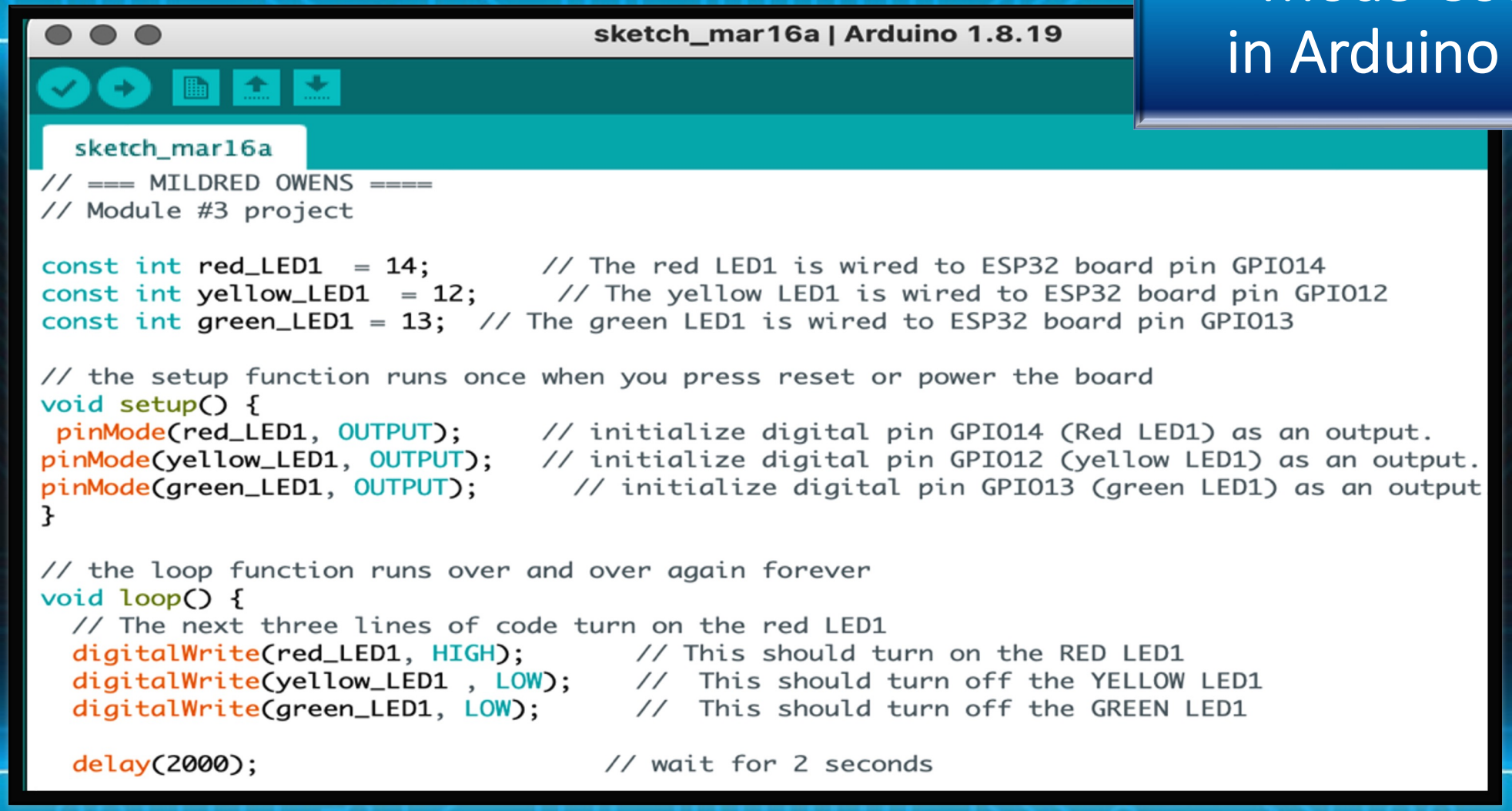
Circuit with working LEDs



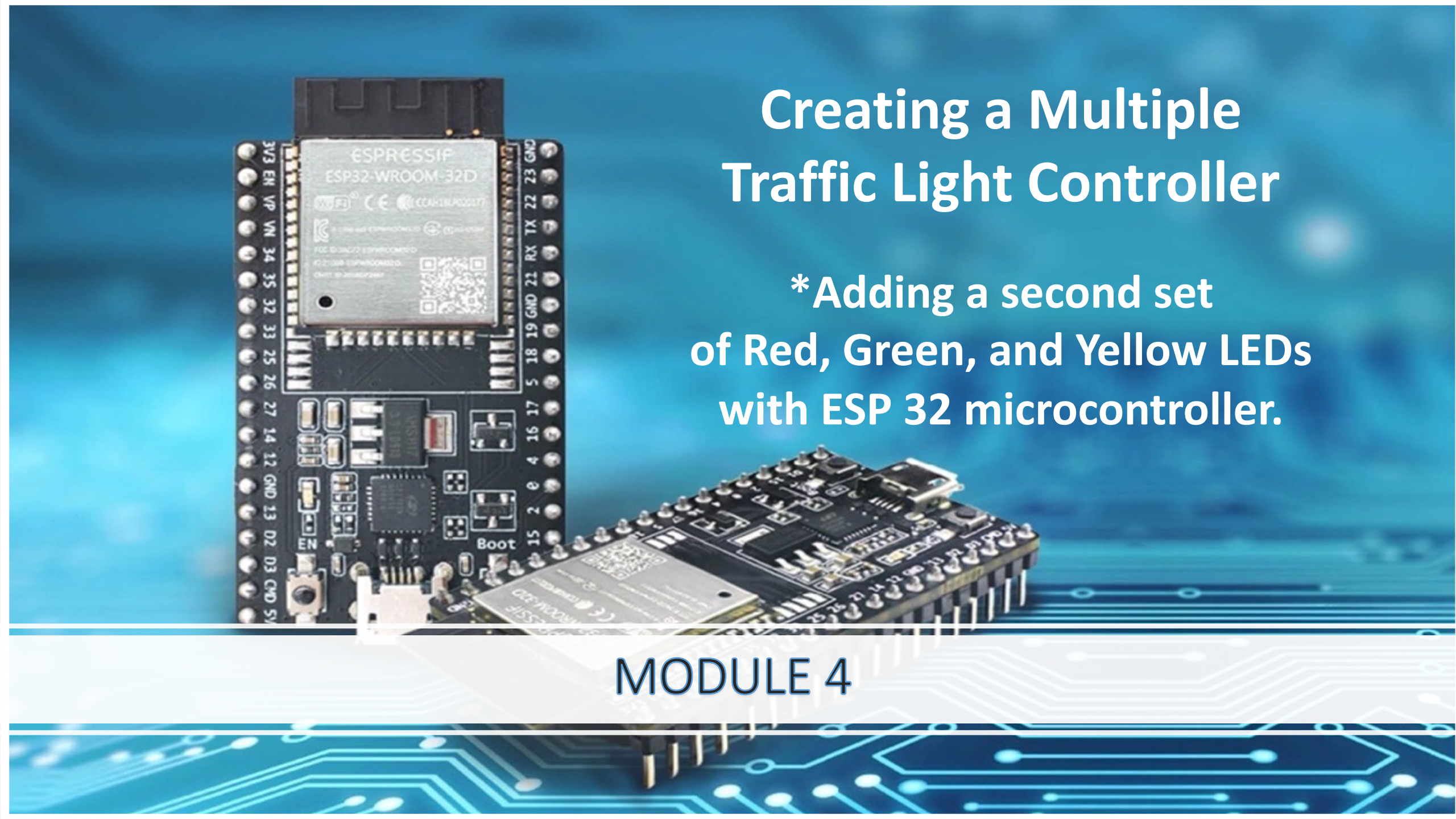
Adding a 220 Ohm Resistors (optional)

- There Red LED is not as bright as it is without the 220 resistor.
- Resistor will extend its life.

Mod3 Code in Arduino IDE




```
// === MILDRED OWENS ====
// Module #3 project

const int red_LED1  = 14;        // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1  = 12;      // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13;  // The green LED1 is wired to ESP32 board pin GPIO13

// the setup function runs once when you press reset or power the board
void setup() {
 pinMode(red_LED1, OUTPUT);      // initialize digital pin GPIO14 (Red LED1) as an output.
pinMode(yellow_LED1, OUTPUT);    // initialize digital pin GPIO12 (yellow LED1) as an output.
pinMode(green_LED1, OUTPUT);      // initialize digital pin GPIO13 (green LED1) as an output
}

// the loop function runs over and over again forever
void loop() {
  // The next three lines of code turn on the red LED1
  digitalWrite(red_LED1, HIGH);        // This should turn on the RED LED1
  digitalWrite(yellow_LED1 , LOW);     //  This should turn off the YELLOW LED1
  digitalWrite(green_LED1, LOW);       //  This should turn off the GREEN LED1

  delay(2000);                        // wait for 2 seconds
```

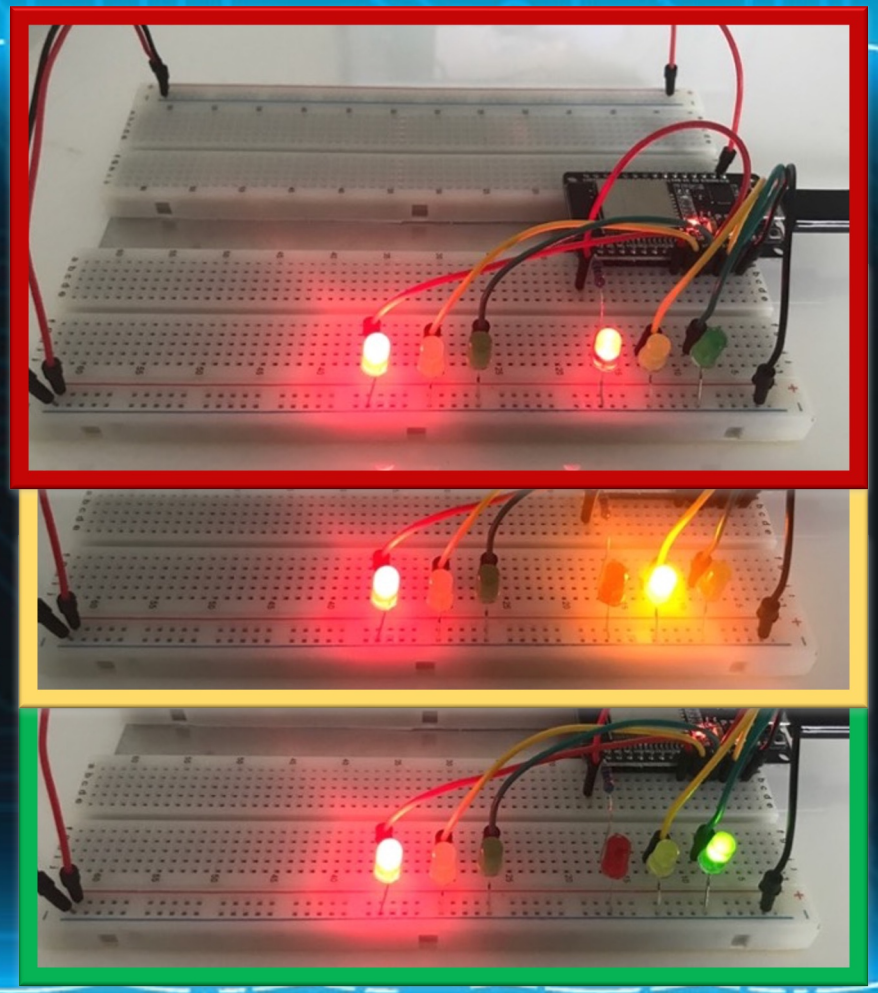


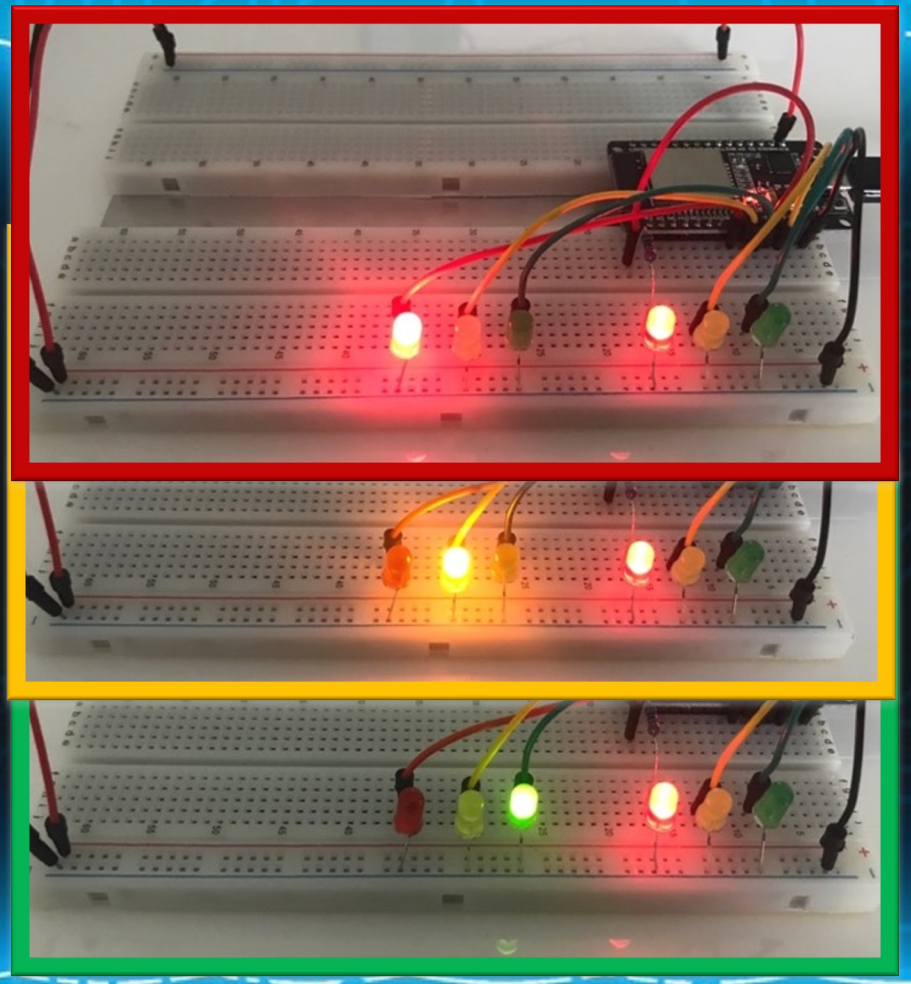
Creating a Multiple Traffic Light Controller

***Adding a second set of Red, Green, and Yellow LEDs with ESP 32 microcontroller.**

MODULE 4

Circuit with working LEDs cycling through red - yellow - green





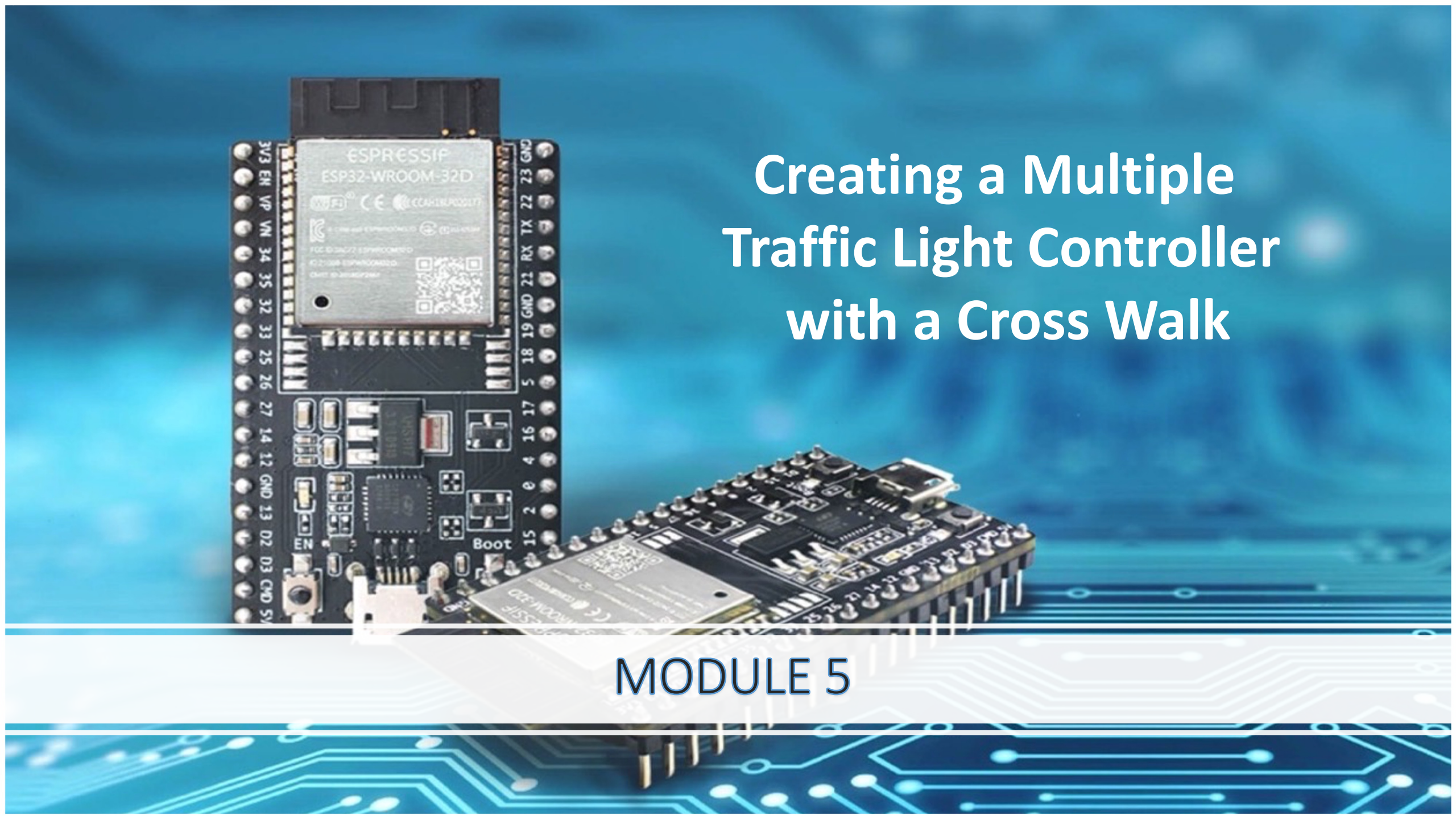
*ESP 32 Board *Colored LEDs: Red, Yellow and Green (two sets)
*220 Ohm Resistors(optional) *Wires *Breadboard

Mod 4 Code in Arduino IDE

Module_4_mar23a | Arduino 1.8.19

Module_4_mar23a

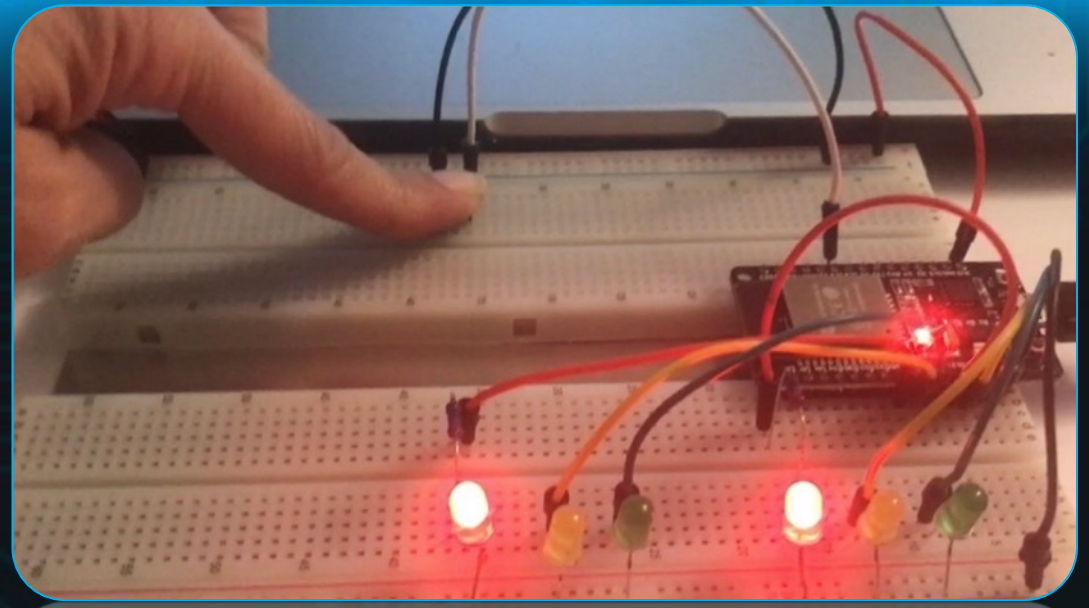
```
// === MILDRED OWENS ====
// Module #4 project

// Define some labels
const int red_LED1  = 14;   // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1  =12;   // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
const int red_LED2  = 25;   // The red LED2 is wired to Mega board pin GPIO25
const int yellow_LED2  = 26;   // The yellow LED2 is wired to Mega board pin GPIO 26
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27

// the setup function runs once when you press reset or power the board
void setup() {
 pinMode(red_LED1, OUTPUT);  // initialize digital pin GPIO14 (Red LED1) as an output.
 pinMode(yellow_LED1, OUTPUT);  // initialize digital pin GPIO12 (yellow LED1) as an output.
 pinMode(green_LED1, OUTPUT);    // initialize digital pin GPIO13 (green LED1) as an output.
 pinMode(red_LED2, OUTPUT);  // initialize digital pin GPIO25(Red LED2) as an output.
 pinMode(yellow_LED2, OUTPUT); // initialize digital pin GPIO26 (yellow LED2) as an output.
 pinMode(green_LED2, OUTPUT); // initialize digital pin GPIO27 (green LED2) as an output.
}
```



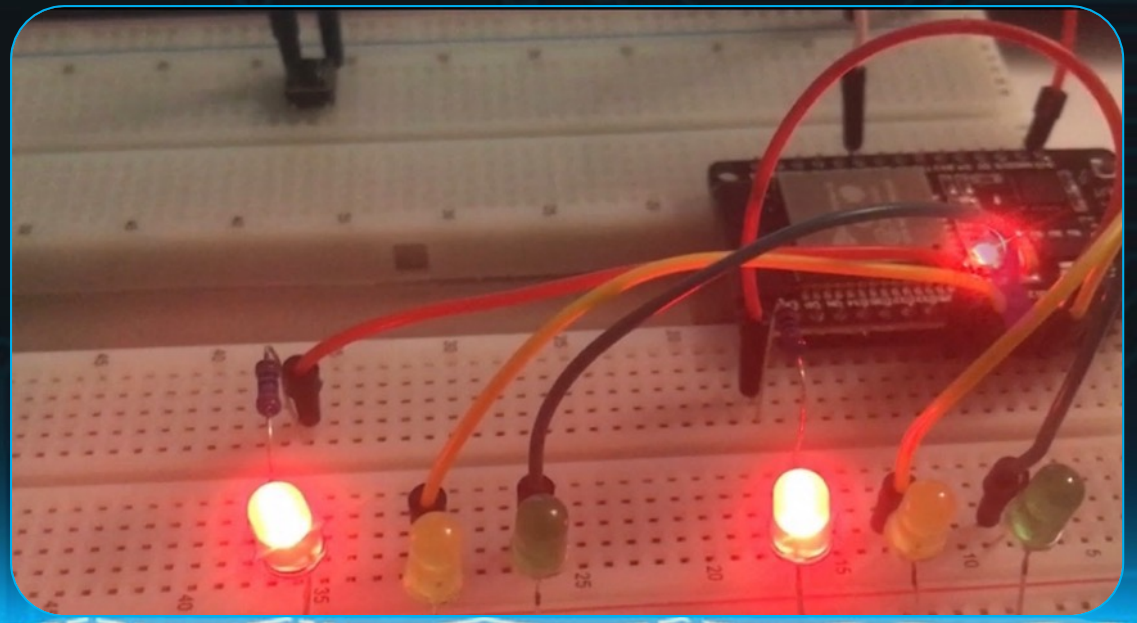
Creating a Multiple Traffic Light Controller with a Cross Walk

MODULE 5

Circuit with working LEDs



- 220 Ohm Resistors (optional)
- Push Button
- Wires
- Breadboard

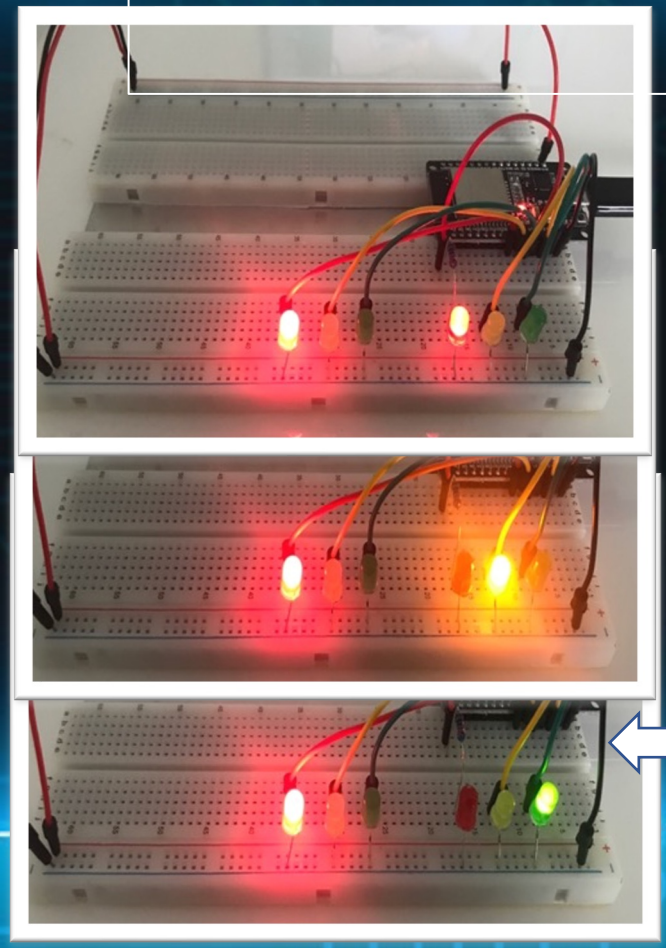


- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)

Pushing button triggers 2 red Leds to flash

STAGES : Open Switch - Close Switch State Cycle

LEDs cycling through red-yellow-green > WALK (10 counts)



/dev/cu.SLAB_USBtoUART

```
 == Do Not Walk ==
 == Do Not Walk ==
Count =  10  == Walk ==
Count =  9  == Walk ==
Count =  8  == Walk ==
Count =  7  == Walk ==
Count =  6  == Walk ==
Count =  5  == Walk ==
Count =  4  == Walk ==
Count =  3  == Walk ==
Count =  2  == Walk ==
Count =  1  == Walk ==
 == Do Not Walk ==
 == Do Not Walk ==
 == Do Not Walk ==
 == Do Not Walk ==
Count =  10  == Walk ==
Count =  9  == Walk ==
Count =  8  == Walk ==
Count =  7  == Walk ==
Count =  6  == Walk ==
Count =  5  == Walk ==
Count =  4  == Walk ==
```

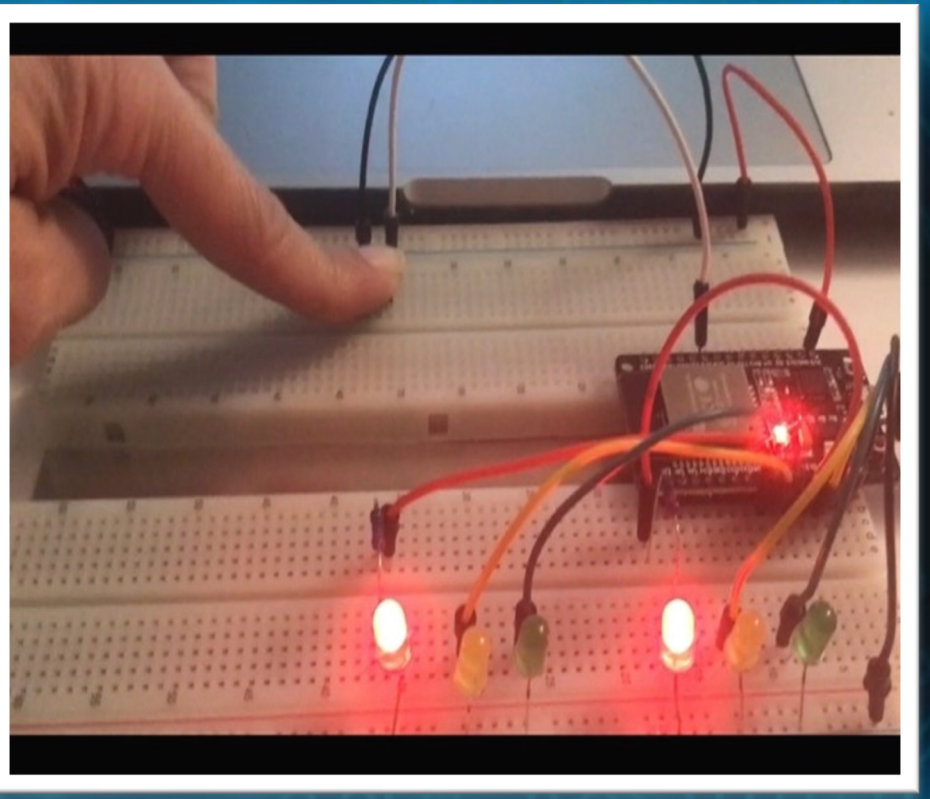
Autoscroll Show timestamp Newline 115200 baud

Open Switch state

Push (Hold) Switch Button “Closed State“ → Working Red LEDs> flashing

- 220 Ohm Resistors (optional)
- Push Button
- Wires
- Breadboard

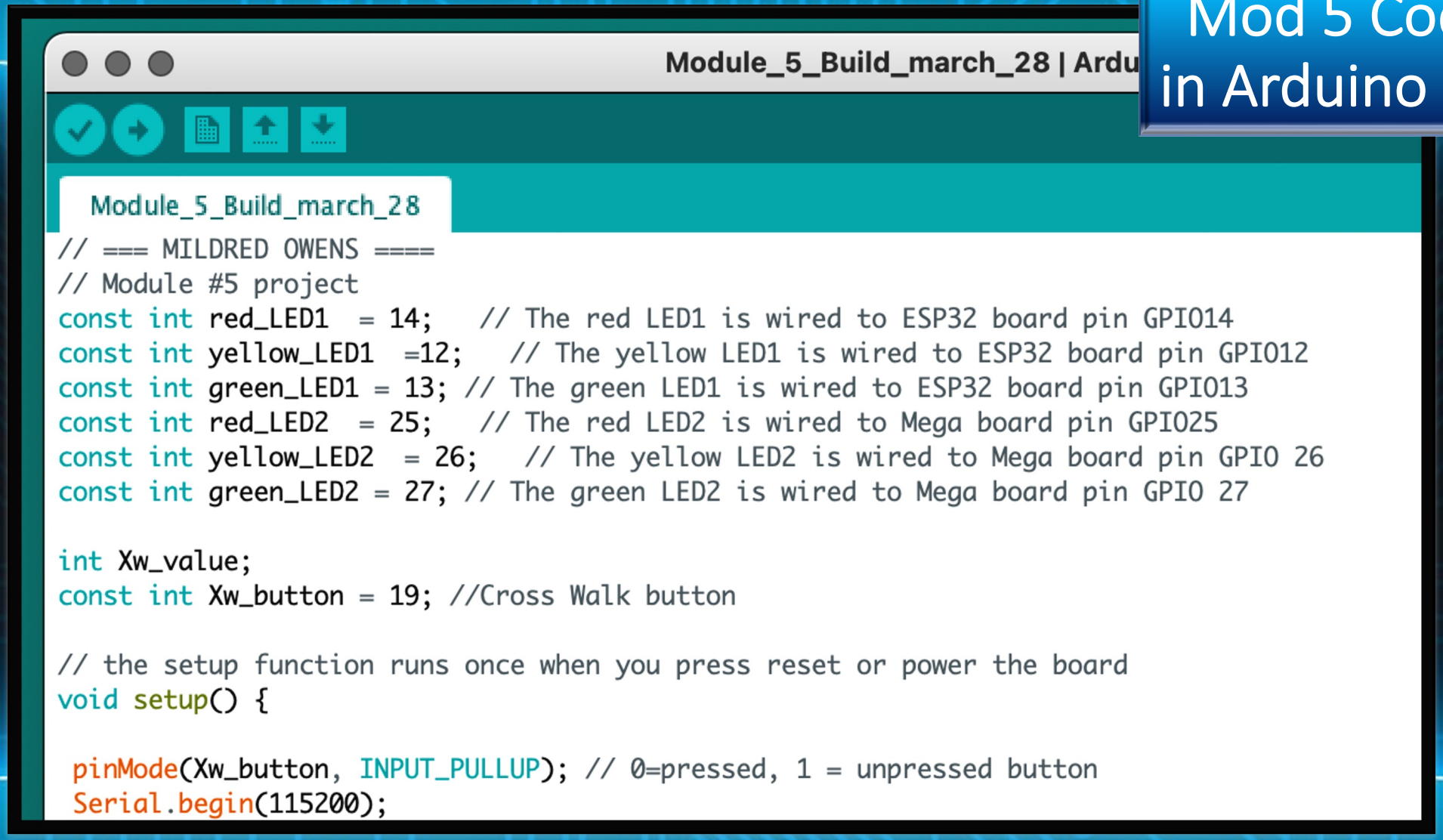
- ESP 32 Board
- Colored LEDs: Red, Yellow and Green (two sets)



```
  == Do Not Walk ==
  == Do Not Walk ==
Count =  10  == Walk ==
Count =  9  == Walk ==
Count =  8  == Walk ==
Count =  7  == Walk ==
Count =  6  == Walk ==
Count =  5  == Walk ==
Count =  4  == Walk ==
Count =  3  == Walk ==
Count =  2  == Walk ==
Count =  1  == Walk ==
  == Do Not Walk ==
  == Do Not Walk ==
  == Do Not Walk ==
  == Do Not Walk ==
Count =  10  == Walk ==
Count =  9  == Walk ==
Count =  8  == Walk ==
```

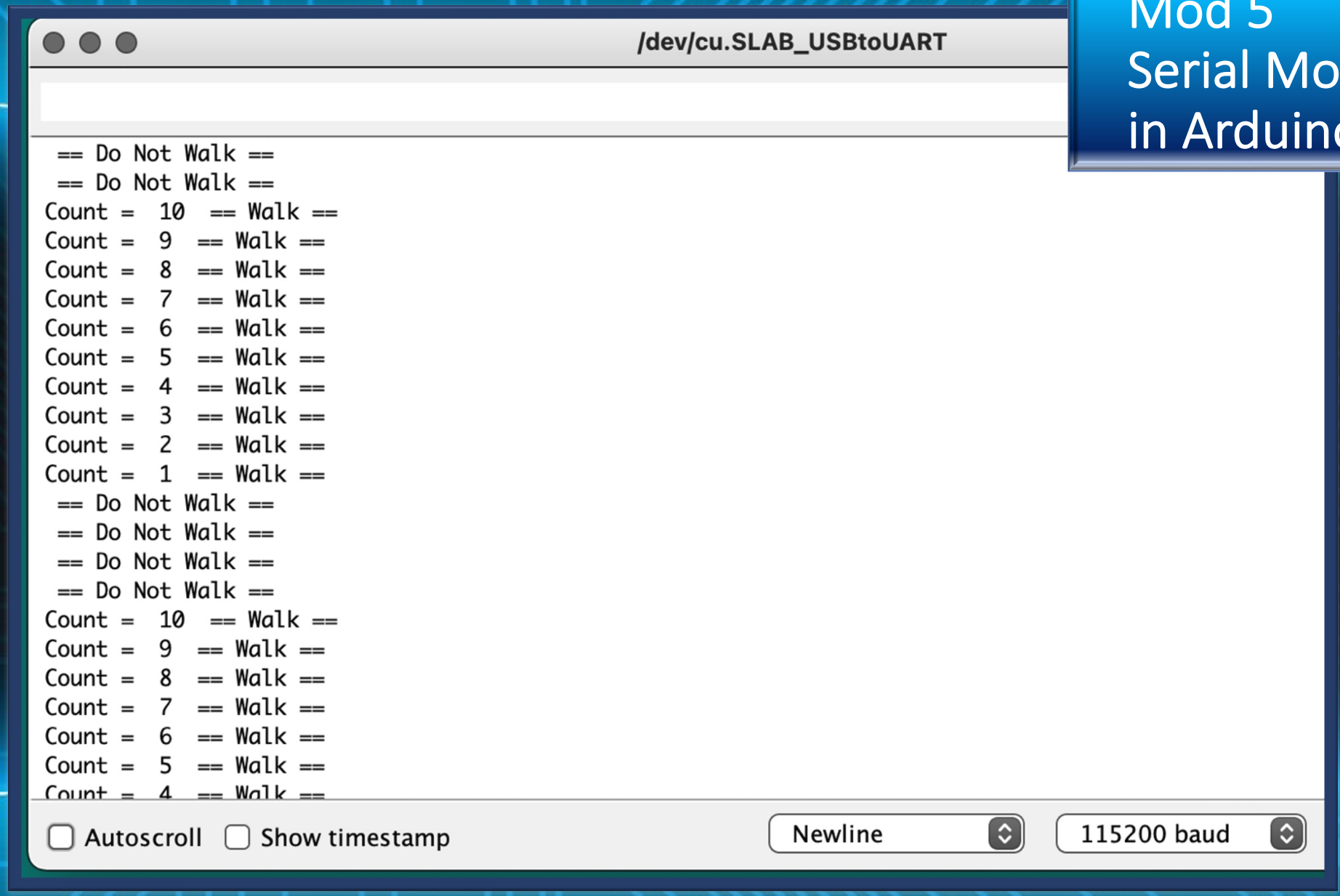
Upon Release of Button > triggers Temporary Switch button > OPEN STATE
R-Y-G Cycle resumes> 10 Counts (Walk)

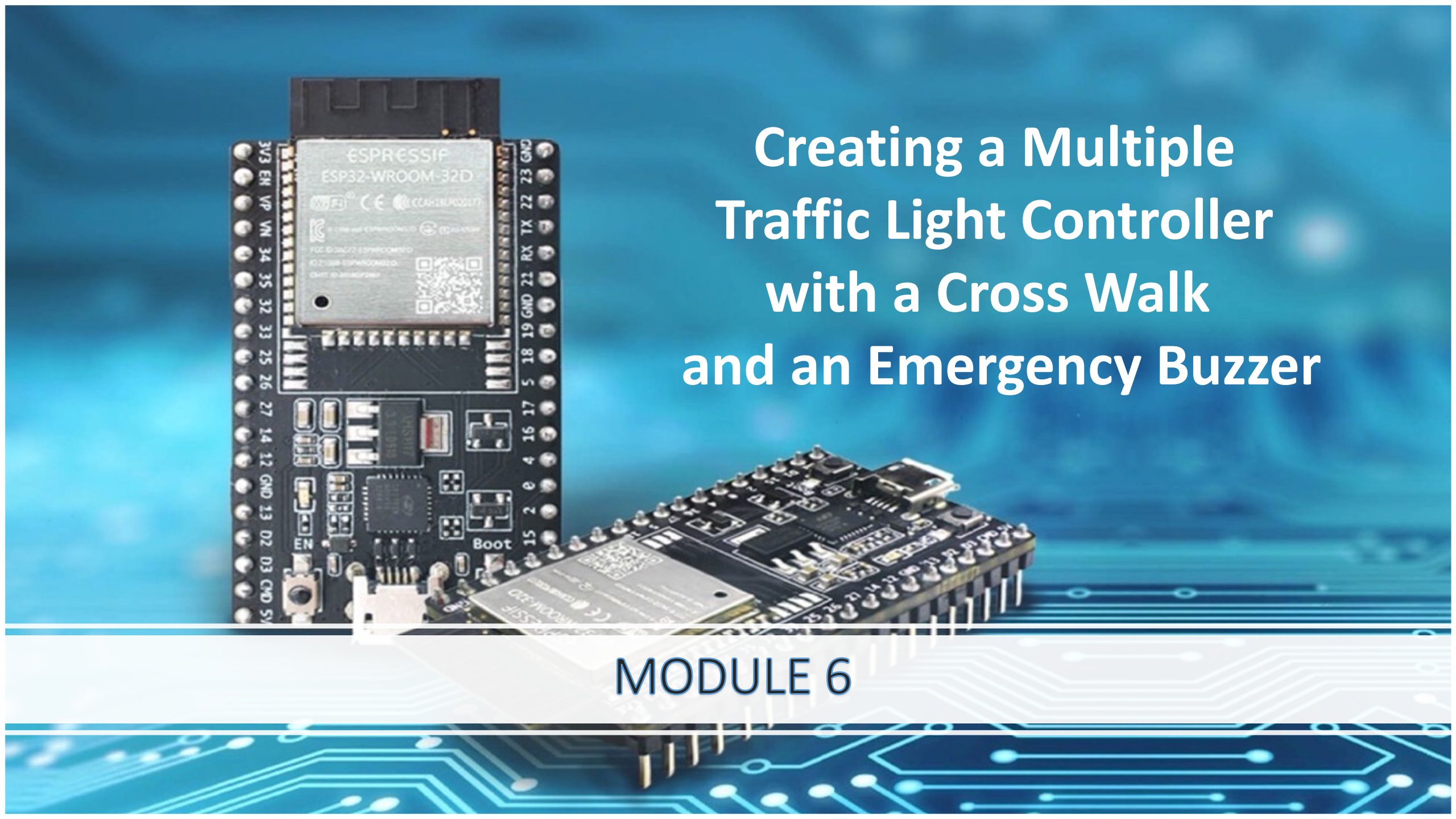
Mod 5 Code in Arduino IDE




```
// === MILDRED OWENS ====
// Module #5 project
const int red_LED1  = 14;   // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1  =12;   // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
const int red_LED2  = 25;   // The red LED2 is wired to Mega board pin GPIO25
const int yellow_LED2  = 26;   // The yellow LED2 is wired to Mega board pin GPIO 26
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27

int Xw_value;
const int Xw_button = 19; //Cross Walk button

// the setup function runs once when you press reset or power the board
void setup() {

 pinMode(Xw_button, INPUT_PULLUP); // 0=pressed, 1 = unpressed button
 Serial.begin(115200);
```

Mod 5
Serial Monitor in Arduino IDE

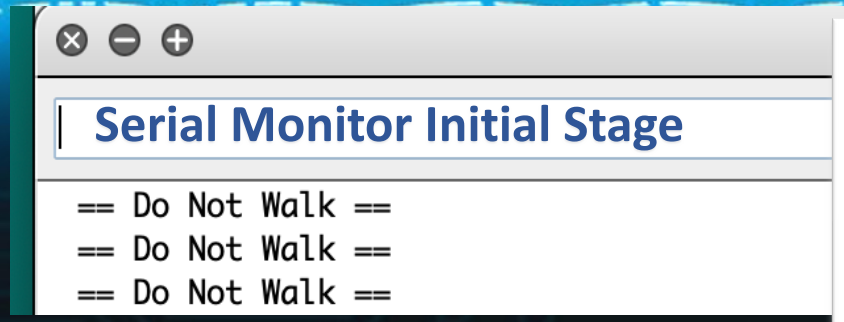




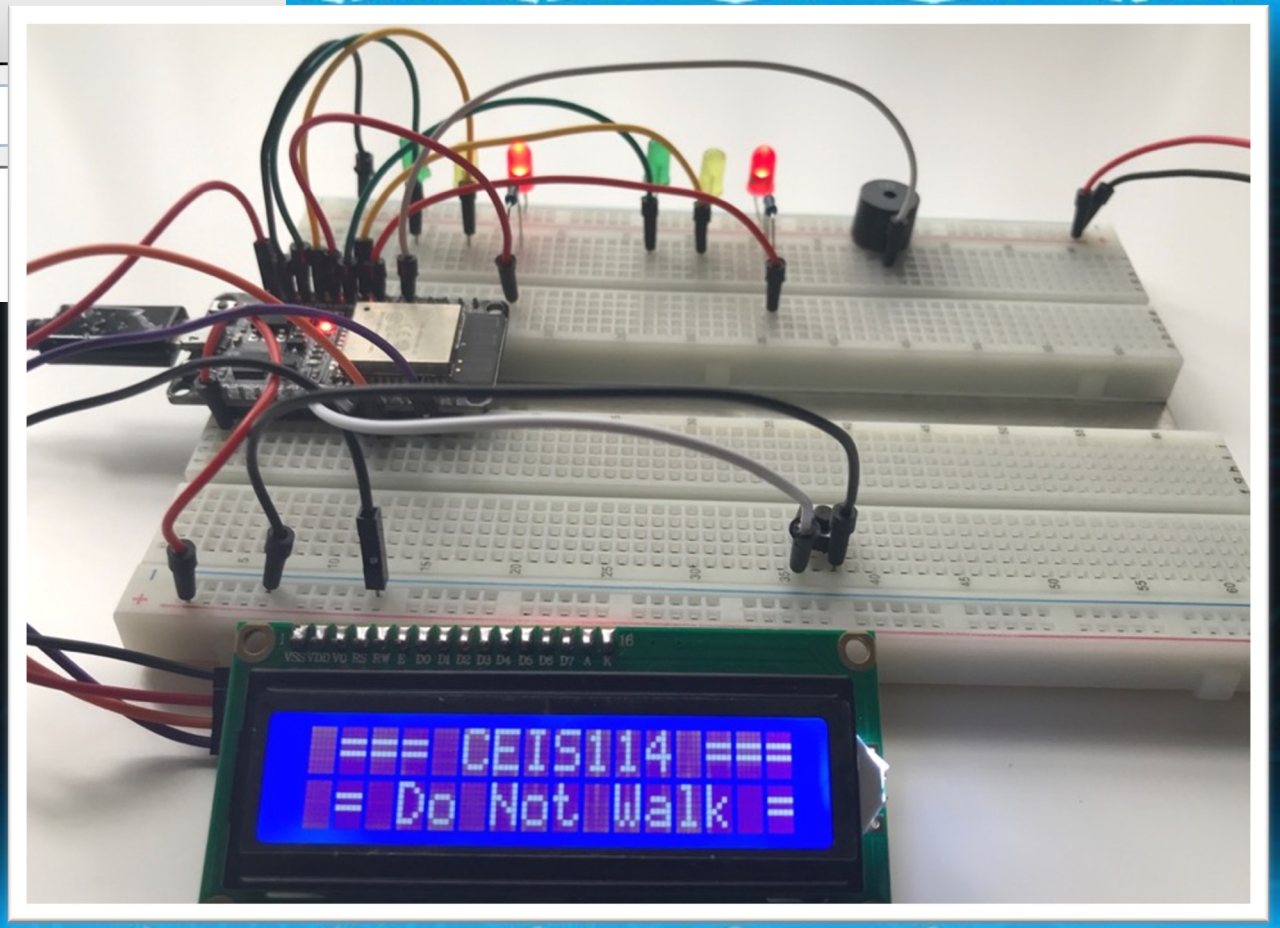
Creating a Multiple Traffic Light Controller with a Cross Walk and an Emergency Buzzer

MODULE 6

Circuit with working LEDs and LCD display



- ESP 32 Board
- Colored LEDs:
- Red, Yellow and Green
- (two sets)
- 220 Ohm Resistors (optional)
- Push Button
- LCD Unit with Message Display
- Wires.
- Breadboard



“Closed State“ Pressing Push Button Switch

Working Red LEDs flashing
DO NOT WALK

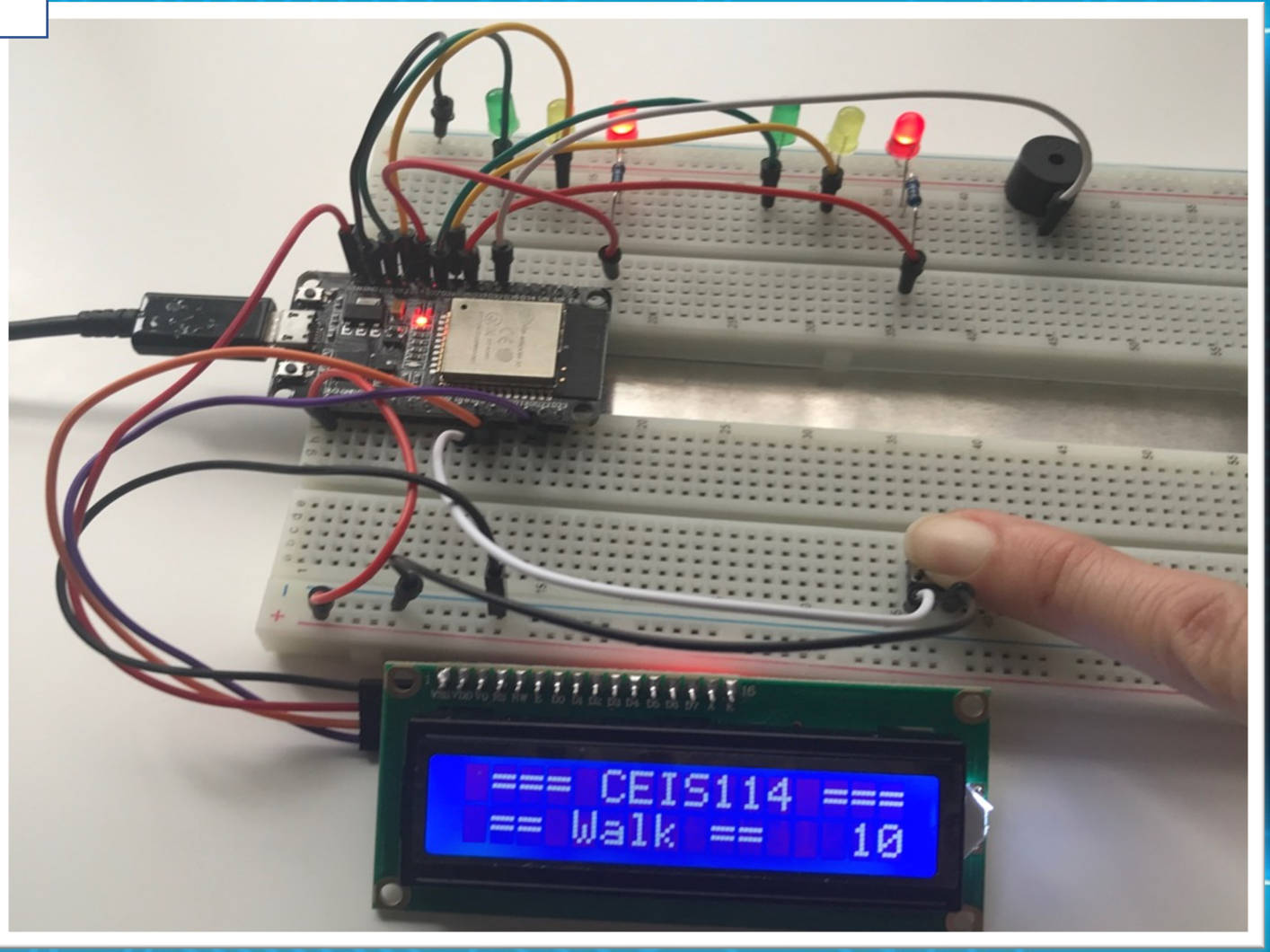
RELEASE BUTTON

Serial Monitor

10 seconds to Walk

```
Count =  10  == Walk ==
Count =  9  == Walk ==
Count =  8  == Walk ==
Count =  7  == Walk ==
Count =  6  == Walk ==
Count =  5  == Walk ==
Count =  4  == Walk ==
Count =  3  == Walk ==
Count =  2  == Walk ==
Count =  1  == Walk ==
Count =  0  == Walk ==
 == Do Not Walk ==
```

Then Resumes Back to Cycle
DO NOT WALK

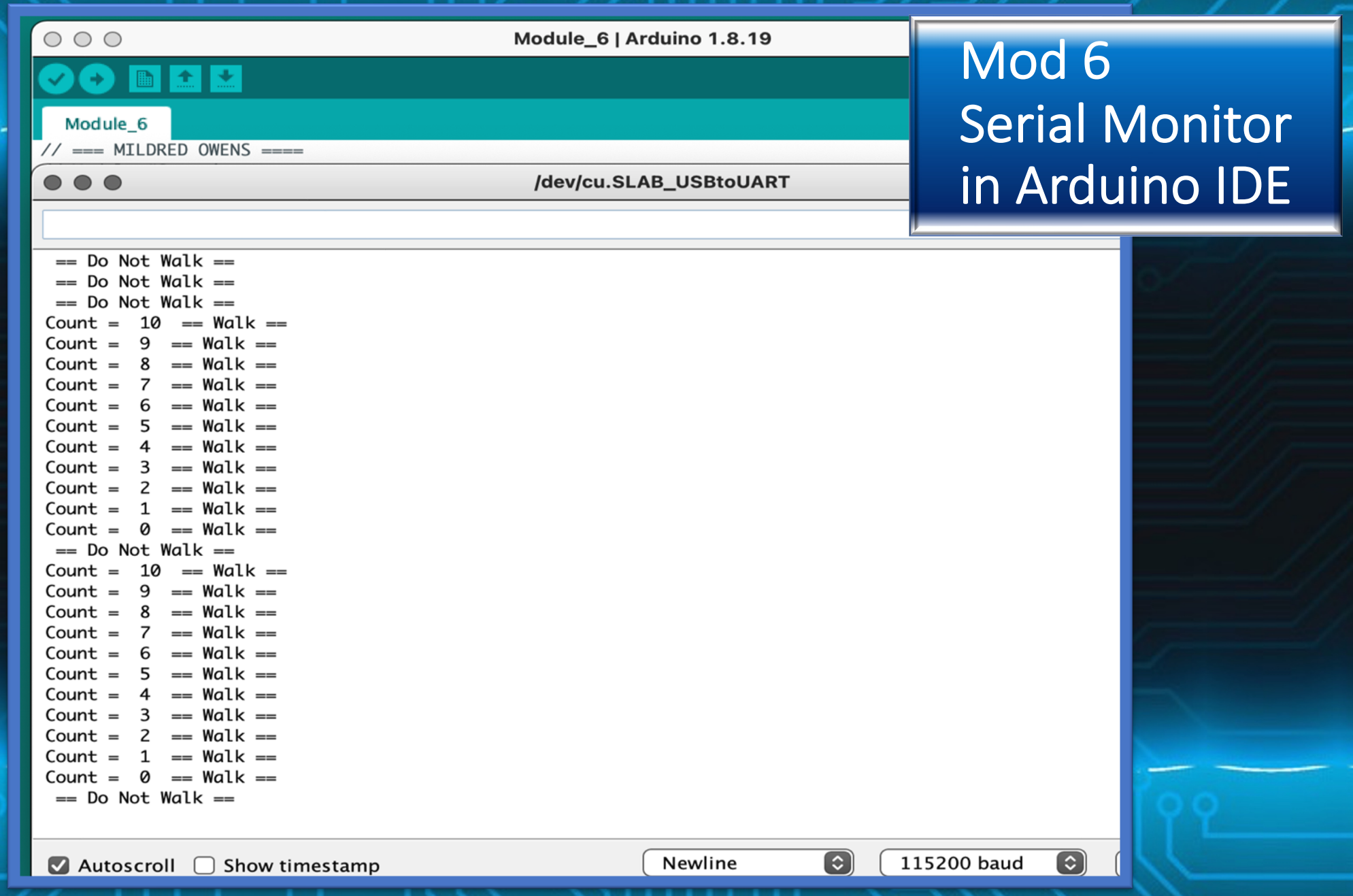


Mod 6 Code in Arduino IDE

Module_6 | Arduino 1.8.19

Module_6

```
// === MILDRED OWENS ====
// Module #6 project
#include <Wire.h>  //lcd
#include <LiquidCrystal_I2C.h> //lcd
LiquidCrystal_I2C lcd(0x27,16,2); //set the LCD address to 0x3F for a 16 chars and 2-line displa
// if it does not work then try 0x3F, if both addresses do not work then run the scan code below

const int bzr=32;       // GPIO32 to connect the Buzzer
//==================== LCD ====================
const int red_LED1  = 14;   // The red LED1 is wired to ESP32 board pin GPIO14
const int yellow_LED1  =12;   // The yellow LED1 is wired to ESP32 board pin GPIO12
const int green_LED1 = 13; // The green LED1 is wired to ESP32 board pin GPIO13
const int red_LED2  = 25;   // The red LED2 is wired to Mega board pin GPIO25
const int yellow_LED2  = 26;   // The yellow LED2 is wired to Mega board pin GPIO 26
const int green_LED2 = 27; // The green LED2 is wired to Mega board pin GPIO 27

int Xw_value;
const int Xw_button = 19; //Cross Walk button

void setup() {
  Serial.begin(115200);
```

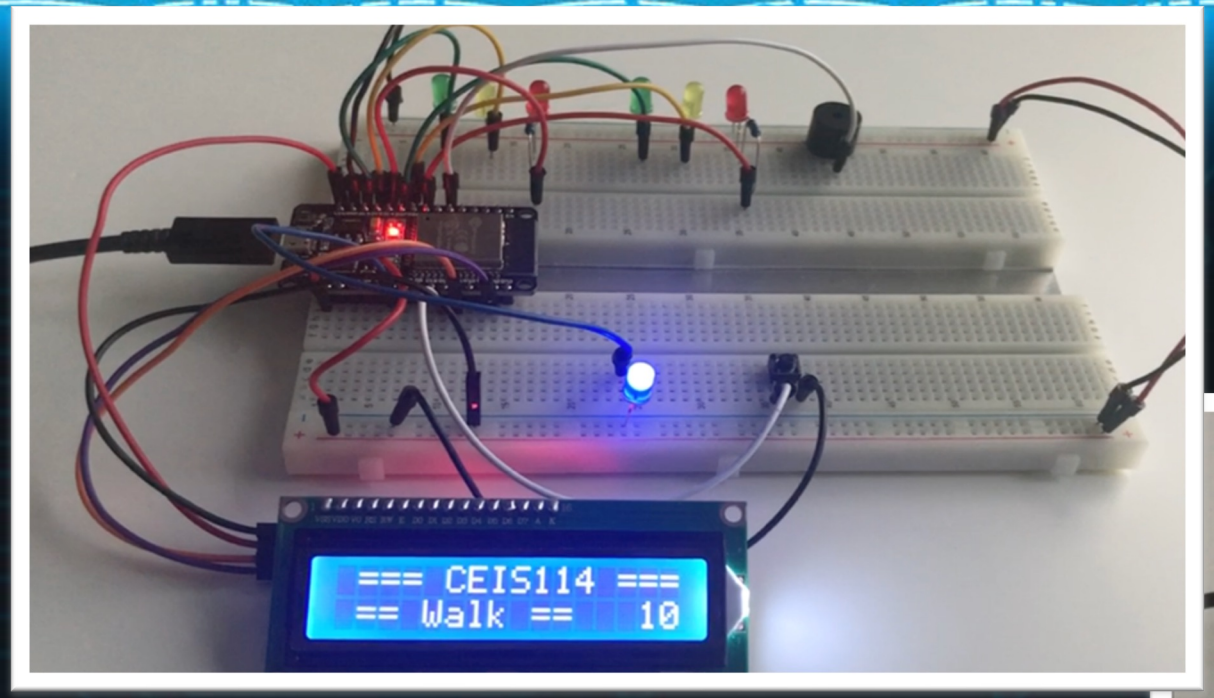


Mod 6
Serial Monitor in Arduino IDE

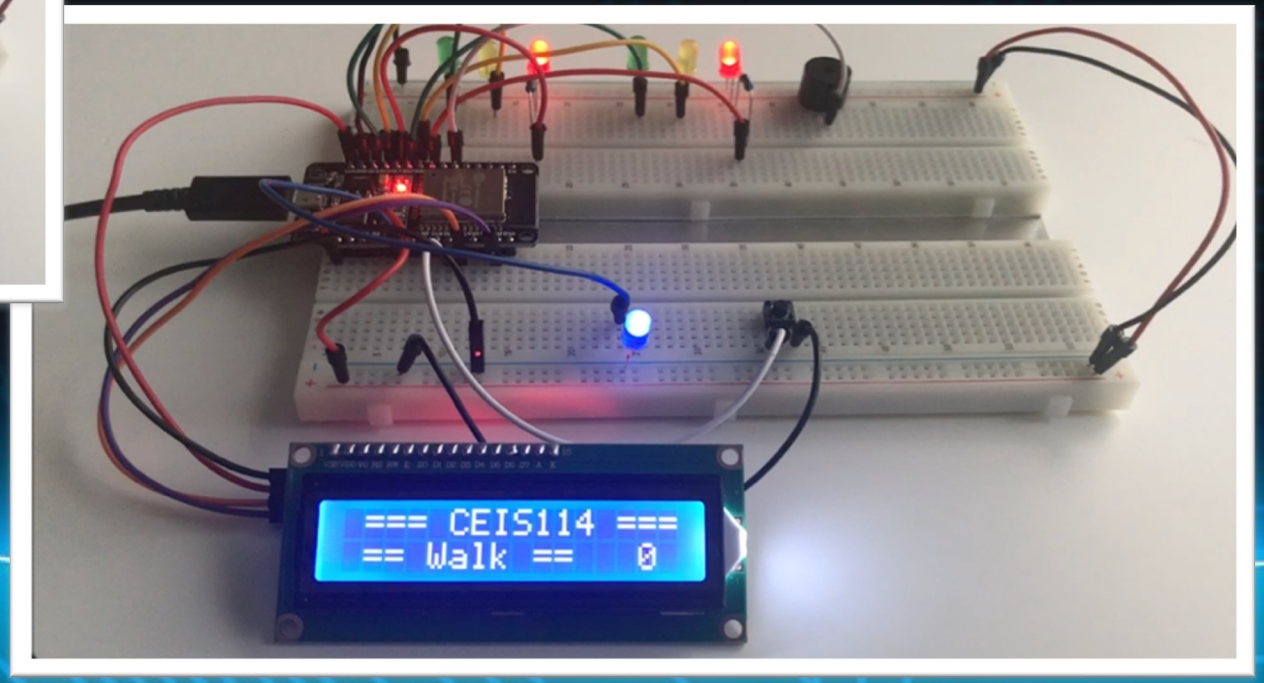
Creating a Multiple Traffic Light Controller with a Cross Walk and an Emergency Buzzer with secured IoT Control via Web (cloud service)

FINAL PROJECT - OPTION 1

Picture of circuit with working LEDs and LCD display



ESP 32 Board
Colored LEDs:
Red, Yellow and Green (two sets)
One Blue LED – Emergency Light
220 Ohm Resistors (optional)

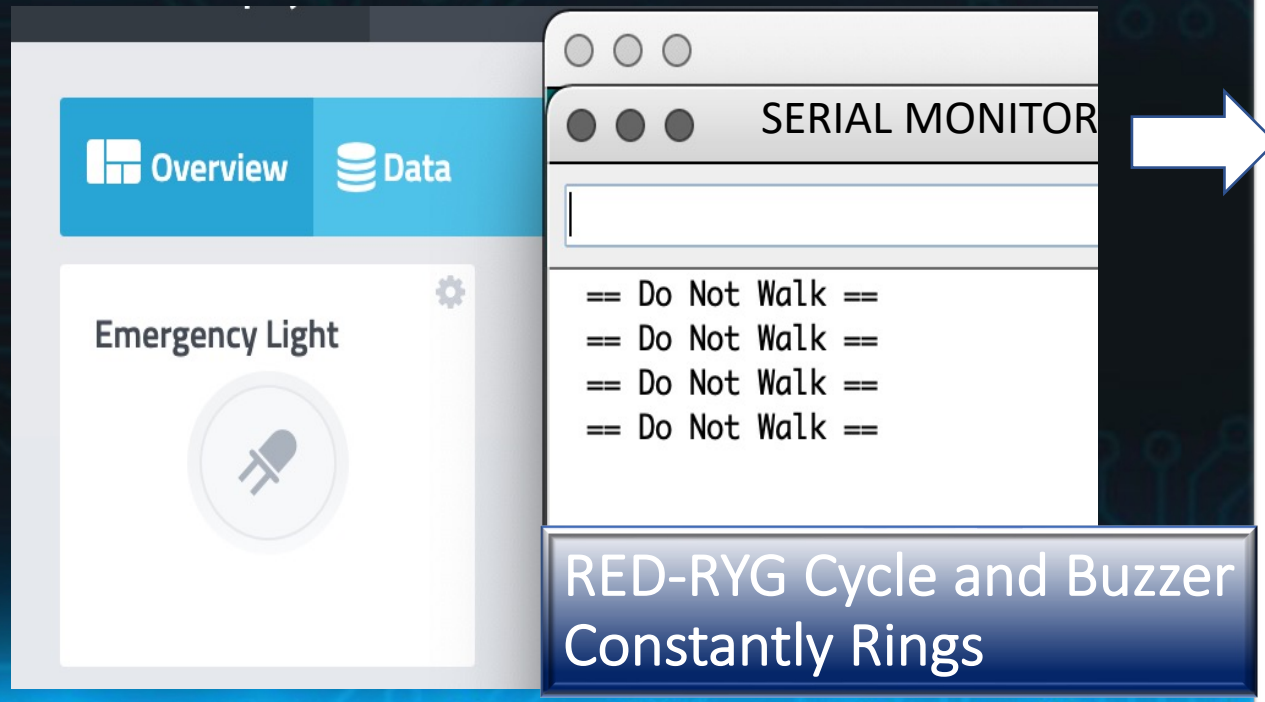


Push Button
LCD Unit
Buzzer
Wires
Breadboard

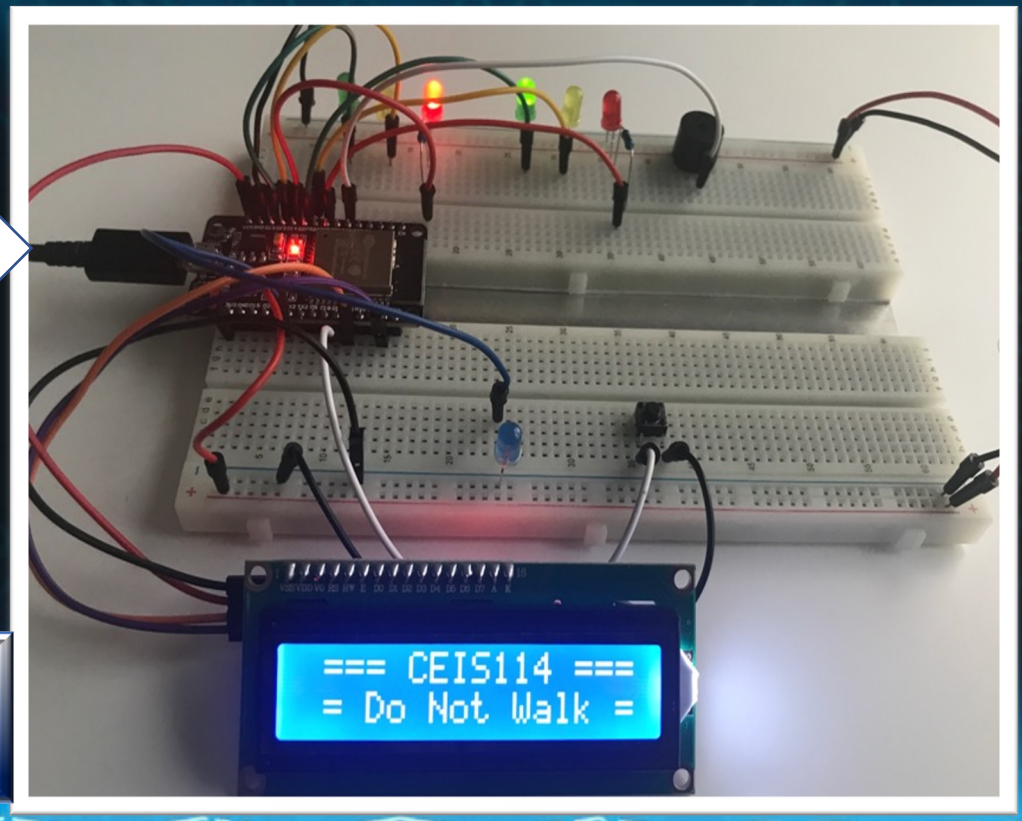
- In this part of the project, the traffic lights are set up by integrating IoT control to the traffic light project using cloud service.
- The controller would give emergency personal access to the traffic light control.
- Upon triggering IoT signal, all traffic from all directions will see red light and a flashing blue light.
- The code wis executed to provided to simulate a single traffic light

STAGES: Serial Monitor in Arduino IDE

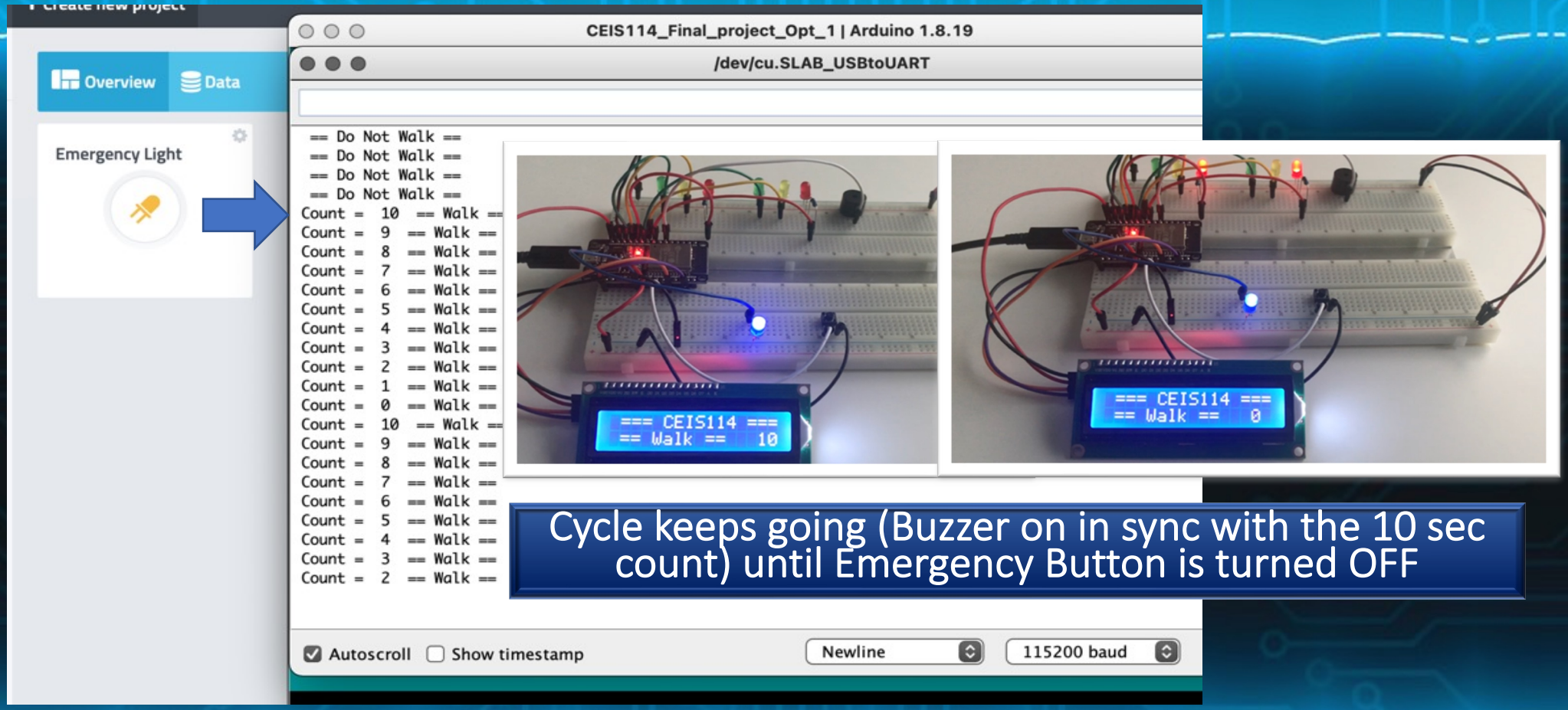
Emergency Button OFF



RED-RYG Cycle and Buzzer Constantly Rings



Emergency Button On> Blue Light Turns on ➡ LCD display WALK : 10 sec.



Cycle keeps going (Buzzer on in sync with the 10 sec count) until Emergency Button is turned OFF

The controller would give emergency personal access to the traffic light control

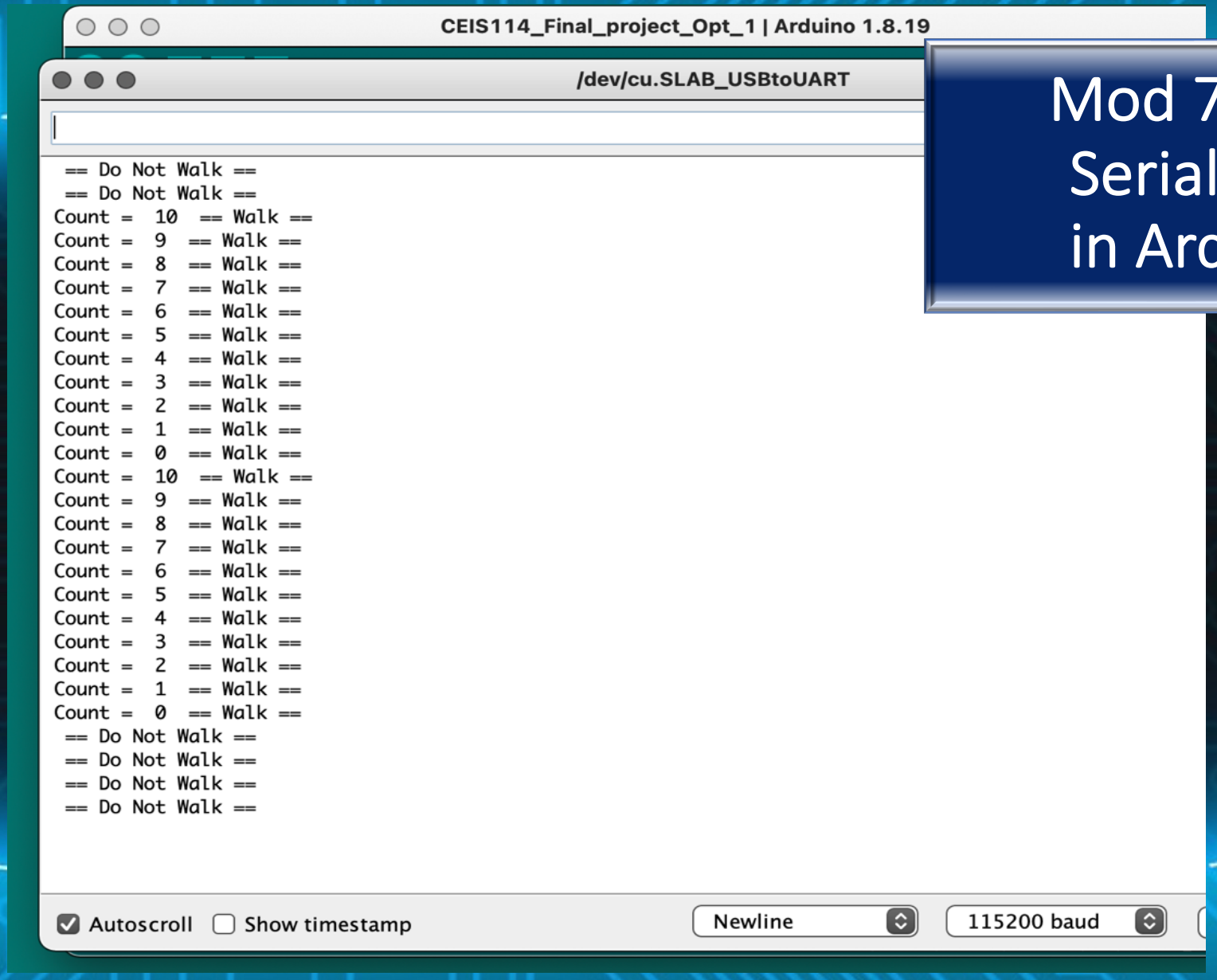
Upon triggering IoT signal, all traffic from all directions will see red light and a flashing blue light.

Mod 7 Option 1 CODE in Arduino IDE

CEIS114_Final_project_Opt_1 | Arduino 1.8.19

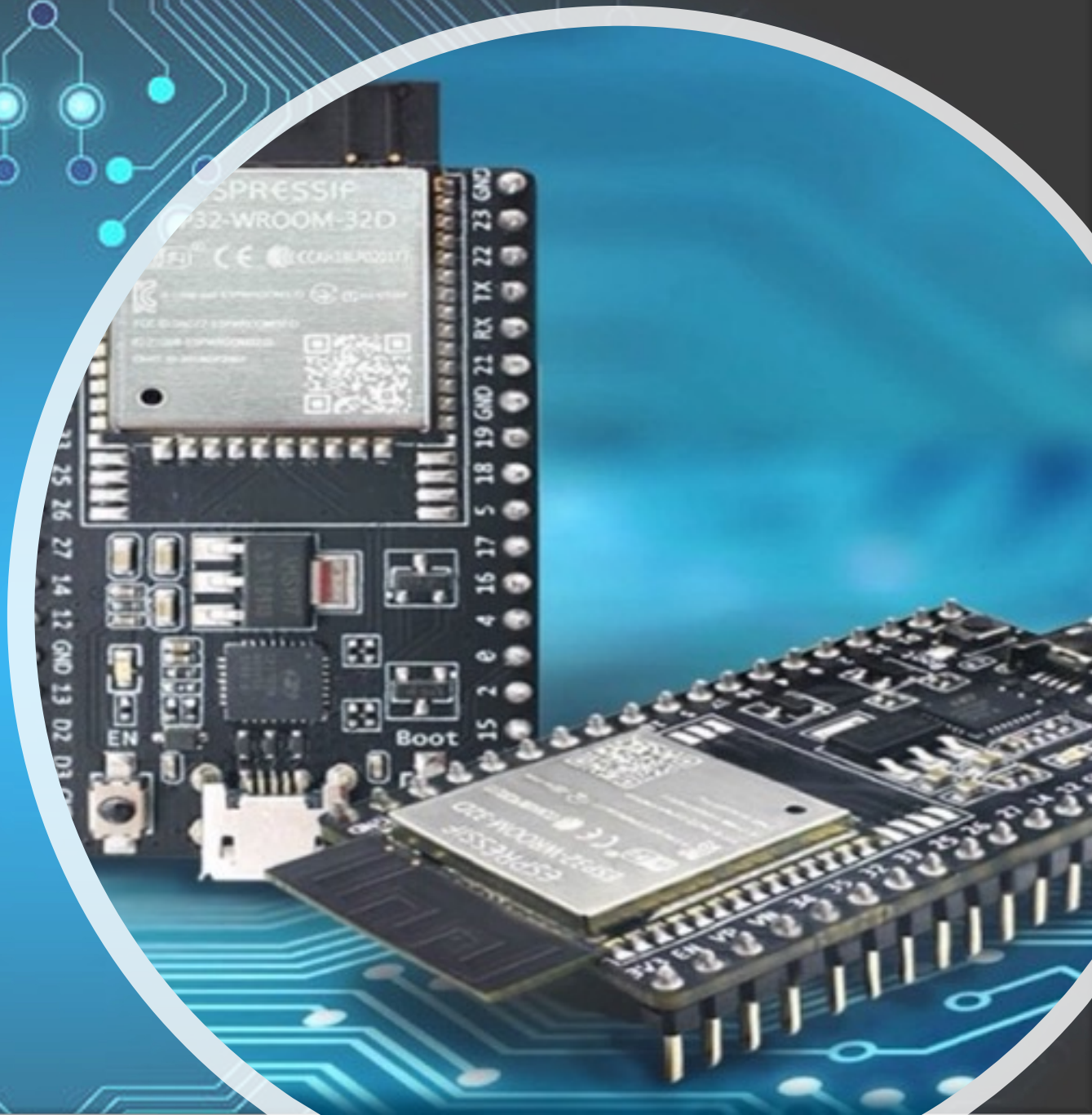
CEIS114_Final_project_Opt_1

```
// === MIDRED OWENS ====
// Final Project Component, Option#1

//#define CAYENNE_DEBUG
#define CAYENNE_PRINT Serial
#include <CayenneMQTTESP32.h>
int ONOFF ;
const int LED0=16;//GPIO16  to trigger the emergency button
// WiFi network info.

 char *ssid = "owens2020";
 char *wifiPassword = "theowens2020";

// Cayenne authentication info. This should be obtained from the Cayenne Dashboard. Replace the
char username[] = "b48bb700-bacd-11ec-9f5b-45181495093e";
char password[] = "7fb9769dd3023b542d72a14643b70247e49a062c";
char clientID[] = "c1df6d70-bacd-11ec-9f5b-45181495093e";
//============== End Cayenne token and SSID/PW Setting ================
//=====================================================================

#include <Wire.h>  //lcd
#include <LiquidCrystal_I2C.h> //lcd
LiquidCrystal_I2C lcd(0x27,16,2); //set the LCD address to 0x3F for a 16 chars and 2-line displa
// if it does not work then try 0x3F, if both addresses do not work then run the scan code below
const int bzr=32;      // GPIO32 to connect the Buzzer
```



Mod 7 Option 1 Serial Monitor in Arduino IDE

CHALLENGES



MODULE 2 CHALLENGE

My MacBook Pro nor my Windows 10 were not identifying my Arduino connection and not showing any com port

TROUBLESHOOT:

1. Checked the connection of my usb –B data cable-intact
2. Replaced and tried again with the other cable from the Kit
   I was thinking/feeling it might be a faulty cord
3. Re-installed drivers, re-tested.

WAS STILL NOT WORKING

Consulted with my study group and found out we were all experiencing the same issue, figured that the 2 micro usb –B cords that came with the kit were not reading data. I tried an old tablet micro usb-B data and charger cord and IT WORKED! Also, had to hit reboot button (which at first, I could not locate) during connection mode.

MODULE 5 CHALLENGE

The 2 Red LEDs for the red stop lights Didn't flash upon pressing the tactile button.

1. Checked placement of button on breadboard:
2. Checked positioning and alignment of wires.
3. Researched on proper positioning of pins on the board
4. Checked the tactile button with the meter. Horizonal and across = 3 buttons were bad. 2 wires faulty.
5. With a good tactile button, upon testing with a meter:
   A. "press – click" on button, it does not keep the circuit on "Close State" ____ (no beep, remains O but remains "Open State" __/ __
   B. If I "press-hold" on the button for a few seconds, It puts the button into the "Close State" with 6 or 7 reading.

SOLUTIONS

1. pins aligned on top/buttom (1 hole in between wires)

T2. To trigger Red LEDs, I had to "push –hold" the button and wait for a few seconds delay while the RYG cycle is on going -until the 2 RED LEDs flash.

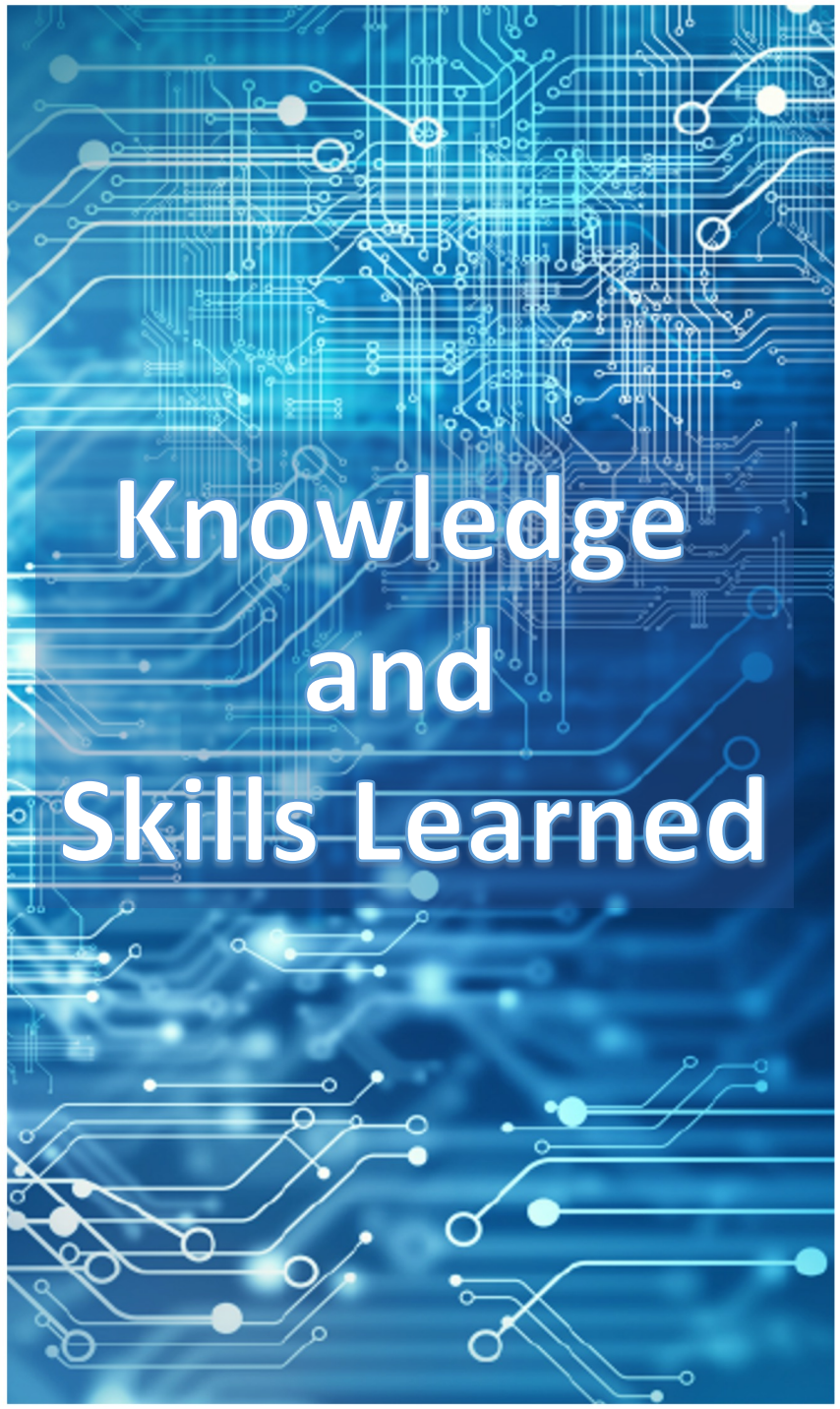
MODULE 6 CHALLENGE

The Buzzer wasn't producing a constant alert sound output but a pulse sound (tic toc tic toc) louder during the 10 sec count down.

Troubleshoot:
I checked all wires. I reviewed the instructional video guide and it instructed that the longer pin or leg would be on the breadboard just as the LEDs. However, my board was the opposite side with blue strip far edge.
I pinned the buzzer's (+) longer leg down my red strip volt (+) and shorter pin on the Bread Board

Issue Solved. The buzzer produced a loud alert sound
LCD: (Do Not Walk); after 10 seconds of delay (RGY cycle)
It switched back to the count down tone.
Then it went back to the buzzer alert on Do Not Walk

- **Identification: Identifying resources needed to build a smart traffic controller.**
  - **Designed solutions for well-defined technology problems and assisted with design of systems, components, or processes appropriate to disciplines such as Information Technology, Information Systems, and or Automation and Controls System.**
- **Testing a system utilizing principles of technology.**
  - **Applied principles of technology in the building, testing, operation, and maintenance of IoT-based systems.**
- **Building: Built a smart traffic light controller utilizing principles of technology.**
  - **Applied knowledge, techniques, skills and modern tools of mathematics, science, engineering, and technology to solve well- defined technology problems appropriate to disciplines such as Information Technology, Information Systems, and or Automation and Controls Systems.**
- **Operating a smart traffic light controller utilizing principles of technology.**
  - **Conducted, analyzed, and interpreted results of standard tests, measurements, and/or experimentation relevant to the field**



- **Described** practical digital devices and systems found in the IoT.
- **Named** the various electronic components needed to create an electronic device.
- **Recognized** the need for numbering systems and Boolean logic in digital devices.
- **Listed** the basic logic circuit building blocks utilized in digital systems.
- **Identified** each system block of an integrated digital device.
- **Explained** the considerations for networking and securing a digital device in the IoT.
- **Explored** the evolving job market in the digitized world.
- **Produced** a basic digital device that could be integrated into the IoT.

CONCLUSION

My venture into field of Tech Core, has been challenging and exciting. CEIS 114 IDD was challenging and fun since I've always wanted to learn basic electronics and understand its principles. CEIS114 gave me basic knowledge and hands on experience of **identifying, testing, building and operating** a simulated **multiple traffic controller system by employing an ESP32 Microcontroller, a Web server, and a Smart Sensor.**

As one aspiring to be a computer specialist someday, majoring in software development and mastering in cybersecurity engineering, I see more clearly how urgent and critical it is to protect information systems and how digitized hardware and network all play together to carry out the goal. The world of science and technology evolved leaps and bounds through the years. Alongside, more challenges arise for security and further breakthroughs are needed both from machine and man, the creative minds behind every machine learning and advancement.

Now I appreciate hardware, software, network and we, all intertwine in the IoT build a better and efficient world of man and machine power.